

MICROSCOPIC_TASK_BREAKDOWN.md

Astitva — SIH26006

Intelligent Freight Forecasting & Vessel Chartering Decision Platform for SAIL

Team: 6 Members | Duration: 48 Hours

PROJECT METADATA

Field	Value
Project Name	Astitva
PS ID	SIH26006
One-Line Description	Intelligent freight forecasting and vessel chartering decision-support platform for SAIL's bulk coal procurement from overseas to East Coast of India
Team Size	6 members
Hackathon Duration	48 hours
Member 1	Ashish — PM + Backend Lead
Member 2	Palak — ML Engineer
Member 3	Om — Backend Developer
Member 4	Prachi — Frontend Developer
Member 5	Param — DevOps + Backend
Member 6	Mahima — Frontend Developer
PM (owns main)	Ashish
Page spec	<code>Features.md</code> — 16 pages. This file's tasks must cover every page in that list.

TEAM ROLES & OWNERSHIP MAP

Member	Role	Primary Responsibility
Ashish	PM + Backend Lead	Project management, JWT auth, analysis routers, admin routers, Phase 2C decision/history endpoints, PR reviews, deployment coordination
Palak	ML Engineer	All ML engines (forecast, feasibility, recommendation, risk, landed cost, stock-out, COA, pooling, regret), data pipeline, model training
Om	Backend Developer	All 17 SQLAlchemy models, Pydantic schemas, 8 data connectors, context engine, scheduler, Phase 2C booking/demand backend
Prachi	Frontend Developer	React/Vite scaffold, auth context, Dashboard, Login, New Analysis pages, Landing/SignUp/ForgotPwd/History pages, Phase 4 frontend deploy
Param	DevOps + Backend	Folder infra, requirements, alembic migrations, DB setup, vendor quotes/map backend, all seed + reset scripts, Render backend deploy
Mahima	Frontend Developer	Analysis Results page (all 11 sections), Route Map, Scenario/Admin/Audit pages, Decision Record/Booking/Demand/Quotes/LiveMap pages

Quick-Reference: Branch → Owner

Branch	Owner(s)
feat/foundation	Param (infra) + Prachi (frontend) + Ashish (backend files)
feat/backend-core	Ashish (auth + routers) + Om (schemas + connectors + engines)
feat/ml-engine	Palak (solo)
feat/frontend-ui	Prachi (Tasks 205–232) + Mahima (Tasks 233–255)
feat/page-features-api	Ashish (Tasks 316, 323–330) + Om (Tasks 317–322, 331–349) + Param (Tasks 350–363)
feat/page-features-ui	Prachi (Tasks 364–386) + Mahima (Tasks 380–383, 387–408)
feat/demo-seed	Param (solo)

Branch	Owner(s)
feat/integration-polish	All Members
feat/deployment	Param (backend) + Prachi (frontend) + Ashish (coordination)

GIT BRANCHING STRATEGY

Three Standing Rules

RULE 1: `main` is ALWAYS deployable. Never push broken code directly to main.
 RULE 2: One active feature branch per member at any time. Never work on main
 RULE 3: Ashish (PM) is the ONLY person who merges into main. Palak opens PRs,

Branch Map

```
main (Ashish owns – always deployable)
├─ feat/foundation      → Param (Phase 0 infra scaffold)
├─ feat/backend-core    → Ashish + Om (FastAPI + Auth + Connectors + Route
├─ feat/ml-engine       → Palak (Forecast + Recommendation + all ML engine
├─ feat/frontend-ui     → Prachi + Mahima (React pages + components)
├─ feat/page-features-api → Ashish + Om + Param (signup, booking, demand, qu
├─ feat/page-features-ui → Prachi + Mahima (16 pages from Features.md)
├─ feat/demo-seed       → Param (demo data + reset script)
└─ feat/integration-polish → All Members (Phase 3 integration + deployment)
```

Full Branch Lifecycle Commands

```
# --- CREATE a new branch (always branch off latest main) ---
git checkout main
git pull origin main
git checkout -b feat/backend-core

# --- DAILY WORK: stage, commit, push ---
git add .
git commit -m "feat(backend): add enrichment connector for bunker prices"
git push origin feat/backend-core

# --- SYNC with main before opening PR (rebase, not merge) ---
git fetch origin
git rebase origin/main
# Fix any conflicts if they appear (see Conflict Playbook below)
git push origin feat/backend-core --force-with-lease

# --- OPEN PR on GitHub ---
# Go to: https://github.com/imstillashish/Astitva/compare/feat/backend-core
# Fill in the PR template (auto-populated from .github/PULL_REQUEST_TEMPLATE.
```

```
# --- AFTER PR IS MERGED: delete local + remote branch ---
git checkout main
git pull origin main
git branch -d feat/backend-core
git push origin --delete feat/backend-core
```

Squash & Merge Strategy

Why squash? Each feature branch may have 20-30 messy WIP commits.
Squash & Merge collapses them into ONE clean commit on main.
This keeps `git log` on main readable – one commit per feature, not 30.

Ashish always uses "Squash and merge" button on GitHub (NOT "Create a merge c

```
# Manual squash merge if doing it from CLI:
git checkout main
git pull origin main
git merge --squash feat/backend-core
git commit -m "feat(backend): complete core enrichment + API layer"
git push origin main
```

High-Risk Conflict Files Table

File	Who Edits It	Conflict Risk
backend/app/main.py	Ashish + Om + Param (routers registered here)	 Very High
backend/requirements.txt	Ashish + Om + Palak + Param (each installs packages)	 Very High
backend/app/models/schemas.py	Ashish + Om (shared Pydantic schemas)	 Very High
frontend/src/App.tsx	Prachi + Mahima (routes defined here)	 Medium
frontend/src/types/index.ts	Prachi + Mahima (shared TypeScript types)	 Medium
frontend/src/api/client.ts	Prachi + Mahima (shared API calls)	 Medium
.env.example	Both (add new env vars)	 Low
backend/app/config.py	Mostly Ashish	 Low

Conflict Resolution Playbook

Scenario A — Rebase Conflict (<<< HEAD markers)

```
# You ran: git rebase origin/main
# And you see this in a file:
#
# <<<<<<< HEAD
# router.include_router(forecast_router)  ← YOUR code (your branch)
# =====
# router.include_router(enrichment_router) ← THEIR code (main)
# >>>>>>> origin/main
#
# HOW TO FIX:
# Keep BOTH lines. Delete the <<< === >>> markers. Result:
# router.include_router(forecast_router)
# router.include_router(enrichment_router)
#
# Then:
git add backend/app/main.py
git rebase --continue
```

Scenario B — requirements.txt Conflict (keep both packages)

```
# You see:
# <<<<<<< HEAD
# lightgbm==4.3.0
# =====
# apscheduler==3.10.4
# >>>>>>> origin/main
#
# HOW TO FIX: Keep BOTH lines, alphabetical order preferred:
# apscheduler==3.10.4
# lightgbm==4.3.0
#
git add backend/requirements.txt
git rebase --continue
# Then reinstall: pip install -r backend/requirements.txt
```

Scenario C — main.py Router Conflict (keep both include_router lines)

```
# You see duplicate or conflicting include_router calls.
# HOW TO FIX: Keep ALL include_router lines. Never delete one.
# Correct final state:
# app.include_router(auth_router, prefix="/auth")
# app.include_router(analyses_router, prefix="/analyses")
# app.include_router(enrichment_router, prefix="/enrichment")
# app.include_router(admin_router, prefix="/admin")
#
git add backend/app/main.py
git rebase --continue
```

Scenario D — TSX/JSX File Conflict (keep both component wrappers)

```
# You see conflicting component imports or JSX structure in App.tsx.  
# HOW TO FIX: Keep both Route elements. Check that import paths are correct.  
# Never delete a Route that someone else added.  
# After fixing:  
git add frontend/src/App.tsx  
git rebase --continue
```

Emergency — "I Broke Main"

```
# WARNING: This rewrites history. Only Ashish (PM) should run this.  
# First, find the last good commit hash:  
git log --oneline origin/main  
  
# Then hard-reset main to that commit:  
git checkout main  
git reset --hard <last-good-commit-hash>  
git push origin main --force  
  
# ⚠️ AFTER THIS: all team members must re-clone or reset their local main:  
git checkout main  
git fetch origin  
git reset --hard origin/main
```

PRE-PHASE 0: GITHUB SETUP

Owner: Ashish (PM)

Estimated time: 30 minutes

Do this BEFORE touching any code.

Repository Creation

G1: Open <https://github.com/new> in a browser → (Ashish — PM)

G2: Set Repository name field to `Astitva` → (Ashish — PM)

G3: Set Description field to `Intelligent freight forecasting and vessel chartering decision platform for SAIL – SIH26006` → (Ashish — PM)

G4: Set visibility to **Public** (SIH requires visibility for submission review) → (Ashish — PM)

G5: Ensure "Add a README file" checkbox is **UNCHECKED** (we push our own) → (Ashish — PM)

G6: Ensure "Add .gitignore" dropdown is set to **None** (we write our own) → (Ashish —

PM)

G7: Ensure "Choose a license" dropdown is set to **None** → (Ashish — PM)

G8: Click "Create repository" green button → (Ashish — PM)

✓ VERIFICATION CHECKPOINT

- ☐ Repository URL is `https://github.com/imstillashish/Astitva`
- ☐ Repository is empty (no files, no commits)
- ☐ Visibility shows "Public"

Add Collaborators

G9: Navigate to `Settings → Collaborators and teams` on the repository → (Ashish — PM)

G10: Click "Add people" → search and invite Palak, Om, Prachi, Param, Mahima with role **Write** → click "Add" → (Ashish — PM)

✓ VERIFICATION CHECKPOINT

- ☐ All teammates have received GitHub email invites
- ☐ Teammates accept invitations from their email or notifications
- ☐ Teammates can view and push to <https://github.com/imstillashish/Astitva>

Branch Protection Rules

G11: Navigate to `Settings → Branches` on the repository → (Ashish — PM)

G12: Click "Add branch protection rule" → (Ashish — PM)

G13: Set Branch name pattern field to `main` → (Ashish — PM)

G14: Check "Require a pull request before merging" checkbox → (Ashish — PM)

G15: Set "Required number of approvals" to `1` → (Ashish — PM)

G16: Check "Dismiss stale pull request approvals when new commits are pushed" checkbox → (Ashish — PM)

G17: Check "Allow specified actors to bypass required pull requests" → add Ashish's GitHub username (PM bypass) → (Ashish — PM)

G18: Click "Create" to save the branch protection rule → (Ashish — PM)

✓ VERIFICATION CHECKPOINT

- ☐ Navigate to `Settings → Branches` → main rule shows "Protected"

- ☐ "Required approvals: 1" is shown on the rule

GitHub Labels

G19: Navigate to `Issues → Labels` on the repository → click "New label" → **(Ashish — PM)**

G20: Create label: Name= `feat` , Color= `#0075ca` , Description= `New feature or enhancement` → click "Create label" → **(Ashish — PM)**

G21: Create label: Name= `fix` , Color= `#e4e669` , Description= `Bug fix` → click "Create label" → **(Ashish — PM)**

G22: Create label: Name= `chore` , Color= `#cfd3d7` , Description= `Maintenance, config, setup` → click "Create label" → **(Ashish — PM)**

G23: Create label: Name= `wip` , Color= `#ff8c00` , Description= `work in progress – do not merge` → click "Create label" → **(Ashish — PM)**

G24: Create label: Name= `ready-for-review` , Color= `#008672` , Description= `Ready for PM review and merge` → click "Create label" → **(Ashish — PM)**

VERIFICATION CHECKPOINT

- ☐ Navigate to `Issues → Labels` — all 5 labels visible with correct colors

PR Template

G25: On Ashish's local machine, create folder `.github/` in the project root → **(Ashish — PM)**

G26: Create file `.github/PULL_REQUEST_TEMPLATE.md` with the following exact content:

```
## What does this PR do?
<!-- One sentence describing what this PR adds or changes -->

## Tasks completed
<!-- List task numbers from MICROSCOPIC_TASK_BREAKDOWN.md -->
- [ ] Task N:
- [ ] Task N:

## Verification checklist
- [ ] I ran the backend locally and it starts without errors
- [ ] I ran the frontend locally and it loads without errors
- [ ] I tested the specific feature I built manually in the browser
- [ ] No console errors in the browser developer tools
- [ ] No Python errors or warnings in the terminal

## Screenshots (if frontend change)
```

<!-- Paste a screenshot of what changed in the UI -->

→ (Ashish — PM)

✓ VERIFICATION CHECKPOINT

- ☐ File `.github/PULL_REQUEST_TEMPLATE.md` exists
- ☐ File contains all 5 sections listed above

Initial Repository Push (PM only)

G27: On Ashish's local machine, navigate to the project directory: `cd SIH26006` (or `mkdir Astitva && cd Astitva`) → (Ashish — PM)

G28: Run `git init` in the project root → (Ashish — PM)

G29: Create root `README.md` with project introduction, architecture stack breakdown, and quickstart commands for **Astitva** — SIH26006 → (Ashish — PM)

G30: Configure `.gitignore` with standard rules, explicitly excluding:

- Python virtual environments and caches: `venv/` , `.venv/` , `__pycache__/` , `*.py[cod]`
- Frontend dependencies and builds: `node_modules/` , `dist/` , `.vite/`
- Local database files: `*.db` , `logistics.db` , `backend/logistics.db`
- Internal LLM/planning and scratch files: `.planning/` , `.geminirules` , `MICROSCOPIC_TASK_BREAKDOWN_PROMPT.md`
- Environment variable files: `.env` , `.env.local` , `.env.production`

→ (Ashish — PM)

G31: Run `git add .` to stage `README.md`, `.gitignore`, and `.github/` → (Ashish — PM)

G32: Run `git commit -m "chore(init): initial repository setup with README, gitignore, PR template"` → (Ashish — PM)

G33: Run `git branch -M main` to set default branch to main → (Ashish — PM)

G34: Run `git remote add origin https://github.com/imstillashish/Astitva.git` → (Ashish — PM)

G35: Run `git push -u origin main` → (Ashish — PM)

All Members: Clone and Configure Identity

G36: All 6 members each run on their own machine: `git clone https://github.com/imstillashish/Astitva.git && cd Astitva` → (All Members)

G37: Each member runs: `git config user.name "<YourName>"` → (All Members)

G38: Each member runs: `git config user.email "<your-email>"` → (All Members)

G39: Ashish runs: `git config user.name "Ashish"` → (Ashish — PM + Backend Lead)

G40: Ashish runs: `git config user.email "ashish@<your-email>.com"` → (Ashish — PM + Backend Lead)

✅ VERIFICATION CHECKPOINT

- ☐ All members can run `git remote -v` and see the GitHub URL
- ☐ All members can run `git log --oneline` and see the initial commit
- ☐ All members have their git identity configured

🔥 GIT CHECKPOINT — Remote is live

```
git log --oneline # should show 1 commit: "chore(init): initial repository"
git remote -v    # should show: origin https://github.com/imstillashish
```

✅ All 6 team members are now connected to the same repo. Phase 0 can begin.

🏗️ PHASE 0: FOUNDATION (Hours 0–4) — Param + Prachi + Ashish

Goal: Folder structure, environment files, base configs, and empty-but-runnable app skeleton.

Param creates backend/infra folders and DevOps files. Prachi sets up the React/Vite frontend scaffold. Ashish writes the core backend files (config, main, database). All three work in parallel on `feat/foundation`.

Backend Folder Structure

Owner: Param — DevOps + Backend

Task 1: Run `git checkout main && git pull origin main && git checkout -b feat/foundation` → (Param — DevOps + Backend)

Task 2: Create folder `backend/` in project root → (Param — DevOps + Backend)

Task 3: Create folder `backend/app/` → (Param — DevOps + Backend)

Task 4: Create folder `backend/app/routers/` → (Param — DevOps + Backend)

Task 5: Create folder `backend/app/models/` → (Param — DevOps + Backend)

Task 6: Create folder `backend/app/services/` → (Param — DevOps + Backend)

Task 7: Create folder `backend/app/connectors/` → (Param — DevOps + Backend)

Task 8: Create folder `backend/app/engines/` → (Param — DevOps + Backend)

Task 9: Create folder `backend/app/utils/` → (Param — DevOps + Backend)

Task 10: Create folder `ml/` in project root → (Param — DevOps + Backend)

Task 11: Create folder `ml/models/` → (Param — DevOps + Backend)

Task 12: Create folder `ml/data/` → (Param — DevOps + Backend)

Task 13: Create folder `scripts/` in project root → (Param — DevOps + Backend)

Task 14: Create folder `docs/` in project root → (Param — DevOps + Backend)

✓ VERIFICATION CHECKPOINT

- ☐ Run `find backend/ -type d` — should list all 8 sub-folders
- ☐ Run `find ml/ -type d` — should list `ml/models` and `ml/data`

Backend Base Files

Owner: Ashish — PM + Backend Lead (Tasks 15–23) | Param — DevOps (Tasks 24–27)

Task 15: Create empty file `backend/app/__init__.py` → (Ashish — PM + Backend Lead)

Task 16: Create empty file `backend/app/routers/__init__.py` → (Ashish — PM + Backend Lead)

Task 17: Create empty file `backend/app/models/__init__.py` → (Ashish — PM + Backend Lead)

Task 18: Create empty file `backend/app/services/__init__.py` → (Ashish — PM + Backend Lead)

Task 19: Create empty file `backend/app/connectors/__init__.py` → (Ashish — PM + Backend Lead)

Task 20: Create empty file `backend/app/engines/__init__.py` → (Ashish — PM + Backend Lead)

Task 21: Create empty file `backend/app/utils/__init__.py` → (Ashish — PM + Backend Lead)

Task 22: Create file `backend/app/config.py` — define a `Settings` class using Pydantic `BaseSettings` with the following fields only (no values, just field declarations):

`DATABASE_URL` , `JWT_SECRET_KEY` , `JWT_EXPIRY_MINUTES` , `SHIP_AND_BUNKER_BASE_URL` ,
`BDRY_DATA_SOURCE_URL` , `WORLD_BANK_PINK_SHEET_URL` , `RBI_FOREX_FEED_URL` ,
`WEATHER_API_BASE_URL` , `MODEL_REGISTRY_PATH` , `LOG_LEVEL` , `CORS_ALLOWED_ORIGINS` ,
`ADMIN_UPLOAD_MAX_MB` , `SENTRY_DSN` → **(Ashish — PM + Backend Lead)**

Task 23: Create file `backend/app/main.py` — create a bare FastAPI app instance, add CORS middleware with `allow_origins=["*"]` for hackathon, add a single `GET /health` route returning `{"status": "ok"}` → **(Ashish — PM + Backend Lead)**

Task 24: Create file `backend/requirements.txt` with the following packages (exact versions):

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
pydantic==2.7.1
pydantic-settings==2.2.1
sqlalchemy==2.0.30
alembic==1.13.1
psycpg2-binary==2.9.9
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
httpx==0.27.0
apscheduler==3.10.4
pandas==2.2.2
scikit-learn==1.4.2
lightgbm==4.3.0
statsmodels==0.14.2
feedparser==6.0.11
python-dotenv==1.0.1
```

→ **(Ashish — Backend Lead)**

Task 25: Create file `backend/.env.example` with all 13 env var names from Task 22 listed with empty values (e.g., `DATABASE_URL=`) → **(Param — DevOps + Backend)**

Task 26: Create file `backend/.env` by copying `.env.example` and filling in local development values — this file must NOT be committed (already in `.gitignore`) → **(Param — DevOps + Backend)**

Task 27: Create file `backend/alembic.ini` by running `cd backend && alembic init alembic` — this generates the alembic config and migrations folder → **(Param — DevOps + Backend)**

✓ VERIFICATION CHECKPOINT

- ☐ Run `cd backend && pip install -r requirements.txt` — no errors
- ☐ Run `cd backend && uvicorn app.main:app --reload` — server starts on port 8000

- ☐ Open browser at `http://localhost:8000/health` — shows `{"status": "ok"}`
 - ☐ Open browser at `http://localhost:8000/docs` — FastAPI Swagger UI loads
-

Frontend Folder Structure

Owner: Prachi — Frontend Developer

Task 28: Palak runs: `cd Astitva && npm create vite@latest frontend -- --template react-ts` → **(Prachi — Frontend Developer)**

Task 29: Run `cd frontend && npm install` → **(Prachi — Frontend Developer)**

Task 30: Run `cd frontend && npm install tailwindcss postcss autoprefixer && npx tailwindcss init -p` → **(Prachi — Frontend Developer)**

Task 31: Run `cd frontend && npm install react-router-dom@6 axios recharts react-leaflet leaflet @types/leaflet` → **(Prachi — Frontend Developer)**

Task 32: Create folder `frontend/src/pages/` → **(Prachi — Frontend Developer)**

Task 33: Create folder `frontend/src/components/` → **(Prachi — Frontend Developer)**

Task 34: Create folder `frontend/src/api/` → **(Prachi — Frontend Developer)**

Task 35: Create folder `frontend/src/types/` → **(Prachi — Frontend Developer)**

Task 36: Create folder `frontend/src/hooks/` → **(Prachi — Frontend Developer)**

Task 37: Create folder `frontend/src/utils/` → **(Prachi — Frontend Developer)**

Task 38: Create folder `frontend/src/context/` → **(Prachi — Frontend Developer)**

Task 39: Edit `frontend/tailwind.config.js` — set `content` array to `["./index.html", "./src/**/*.{js,ts,jsx,tsx}"]` → **(Prachi — Frontend Developer)**

Task 40: Edit `frontend/src/index.css` — replace all content with three Tailwind directives: `@tailwind base;`, `@tailwind components;`, `@tailwind utilities;` → **(Prachi — Frontend Developer)**

Task 41: Create file `frontend/.env.example` with: `VITE_API_BASE_URL=http://localhost:8000` → **(Prachi — Frontend Developer)**

Task 42: Create file `frontend/.env` with: `VITE_API_BASE_URL=http://localhost:8000` → **(Prachi — Frontend Developer)**

Task 43: Edit `frontend/vite.config.ts` — add `server.proxy` config to proxy `/api` to `http://localhost:8000` to avoid CORS issues in development → **(Prachi — Frontend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Run `cd frontend && npm run dev` — Vite server starts on port 5173
- ☐ Open browser at `http://localhost:5173` — default Vite page loads (no errors in console)
- ☐ Tailwind is working: add a `className="text-red-500"` temporarily to `App.tsx`, verify text turns red

Shared Types File

Owner: Prachi — Frontend Developer

Task 44: Create file `frontend/src/types/index.ts` — declare the following TypeScript interfaces (empty bodies for now, to be filled in later phases): `Analysis`, `ContextObject`, `ForecastResult`, `FeasibilityResult`, `RiskResult`, `RecommendationResult`, `ExplainabilityResult`, `DecisionRecord`, `LandedCost`, `StockOutAlert`, `PoolingResult`, `RegretScore`, `DisruptionAlert` → (Prachi — Frontend Developer)

Task 45: Create file `frontend/src/api/client.ts` — create an Axios instance with `baseUrl` set to `import.meta.env.VITE_API_BASE_URL` and a request interceptor that reads a `token` from `localStorage` and adds it as `Authorization: Bearer <token>` header → (Prachi — Frontend Developer)

Database Setup

Owner: Param — DevOps + Backend

Task 46: Create a local PostgreSQL database named `astitva_dev` using: `createdb astitva_dev` (or via pgAdmin/DBever) → (Param — DevOps + Backend)

Task 47: Set `DATABASE_URL` in `backend/.env` to `postgresql://postgres:<password>@localhost:5432/astitva_dev` → (Param — DevOps + Backend)

Task 48: Create file `backend/app/database.py` — define SQLAlchemy `engine` using `DATABASE_URL` from config, define `SessionLocal` `sessionmaker`, define `Base` declarative base, define `get_db` dependency function yielding a session → (Param — DevOps + Backend)

✓ VERIFICATION CHECKPOINT

- ☐ Run `cd backend && python -c "from app.database import engine; print(engine)"` — prints engine without error
- ☐ PostgreSQL connection succeeds (no password/connection errors)

App Router Setup

Owner: Prachi — Frontend Developer

Task 49: Delete the default `frontend/src/App.tsx` content and replace with a bare `BrowserRouter` wrapping a `Routes` component containing placeholder `Route` elements for paths: `/login`, `/dashboard`, `/analyses/new`, `/analyses/:id`, `/analyses/:id/scenarios`, `/admin/reference`, `/admin/users`, `/audit-log`, `*` (catch-all 404) → **(Prachi — Frontend Developer)**

Task 50: Create placeholder page file `frontend/src/pages/LoginPage.tsx` — returns a single `<div>Login Page</div>` → **(Prachi — Frontend Developer)**

Task 51: Create placeholder page file `frontend/src/pages/DashboardPage.tsx` — returns a single `<div>Dashboard</div>` → **(Prachi — Frontend Developer)**

Task 52: Create placeholder page file `frontend/src/pages/NewAnalysisPage.tsx` — returns a single `<div>New Analysis</div>` → **(Prachi — Frontend Developer)**

Task 53: Create placeholder page file `frontend/src/pages/AnalysisResultsPage.tsx` — returns a single `<div>Analysis Results</div>` → **(Prachi — Frontend Developer)**

Task 54: Create placeholder page file `frontend/src/pages/ScenarioViewPage.tsx` — returns a single `<div>Scenario View</div>` → **(Prachi — Frontend Developer)**

Task 55: Create placeholder page file `frontend/src/pages/AdminReferencePage.tsx` — returns a single `<div>Admin Reference Data</div>` → **(Prachi — Frontend Developer)**

Task 56: Create placeholder page file `frontend/src/pages/AdminUsersPage.tsx` — returns a single `<div>Admin Users</div>` → **(Prachi — Frontend Developer)**

Task 57: Create placeholder page file `frontend/src/pages/AuditLogPage.tsx` — returns a single `<div>Audit Log</div>` → **(Prachi — Frontend Developer)**

Task 58: Create placeholder page file `frontend/src/pages/NotFoundPage.tsx` — returns a single `<div>404 Not Found</div>` → **(Prachi — Frontend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Run `cd frontend && npm run dev` — no TypeScript compile errors
- ☐ Navigate to `http://localhost:5173/login` — shows "Login Page"
- ☐ Navigate to `http://localhost:5173/dashboard` — shows "Dashboard"

Docs Scaffold

Owner: Param — DevOps + Backend

Task 59: Create file `docs/API_CONTRACT.md` with a title line `# Astitva API Contract`

and a placeholder table of contents listing 10 endpoints (to be filled later): `POST /auth/login`, `GET /analyses`, `POST /analyses`, `GET /analyses/{id}`, `GET /analyses/{id}/export`, `POST /analyses/{id}/decision`, `GET /admin/users`, `POST /admin/users`, `GET /admin/reference`, `GET /audit-logs` → **(Param — DevOps + Backend)**

Task 60: Create file `scripts/seed_reference_data.py` — empty file with a docstring: `"""Seeds reference data: 4 verified ports, 4 vessel classes, cargo types."""` → **(Param — DevOps + Backend)**

Task 61: Create file `scripts/seed_demo_scenarios.py` — empty file with a docstring: `"""Seeds deterministic demo scenarios for the Golden Demo (Newcastle→Paradip, 75,000 MT coking coal)."""` → **(Param — DevOps + Backend)**

Task 62: Create file `scripts/reset_demo_db.py` — empty file with a docstring: `"""Resets demo database to clean seed state for re-running the demo."""` → **(Param — DevOps + Backend)**

GIT CHECKPOINT — Commit & Push

```
git add .
git commit -m "feat(foundation): complete project scaffold – backend + fr
git push origin feat/foundation
```

PR CHECKPOINT — Open Pull Request

- **Branch:** `feat/foundation` → `main`
- **PR Title:** `feat(Phase 0): Project Foundation Scaffold – Ashish + Palak`
- **Before merging, verify:**
 - ☐ Task 27 verified: `uvicorn app.main:app --reload` starts without error 
 - ☐ Task 43 verified: `npm run dev` starts without error 
 - ☐ Task 49 verified: all 9 route paths render their placeholder text 
- Run: `git checkout main && git pull origin main`

MERGE PROCEDURE

```
git checkout main && git pull origin main
git merge --squash feat/foundation
git commit -m "feat(foundation): complete project scaffold"
git push origin main
```

LIKELY CONFLICT FILES:

- `backend/app/main.py` — both members may have edited this simultaneously

 **CONFLICT RESOLUTION:**  Keep ALL route/middleware lines from both

versions. Never delete a line added by the other member.

PHASE 1A: DATABASE MODELS & MIGRATIONS (Hours 4–10) — Om + Param

Goal: Define all 17 database tables, run migrations, verify schema is correct. Om writes the SQLAlchemy models and seed functions. Param runs the alembic migrations.

SQLAlchemy ORM Models

Task 63: Create file `backend/app/models/user.py` — define SQLAlchemy `User` model with columns: `id` (UUID PK), `email` (String, unique, not null), `password_hash` (String, not null), `role` (String, not null, one of: ADMIN, PROCUREMENT_OFFICER, PLANT_MANAGER, FINANCE_USER, AUDITOR), `is_active` (Boolean, default True), `created_at` (DateTime, server_default now), `updated_at` (DateTime, onupdate now) → (Om — Backend Developer)

Task 64: Create file `backend/app/models/reference_port.py` — define SQLAlchemy `ReferencePort` model with columns: `id` (Integer PK autoincrement), `port_name` (String, unique), `locode` (String), `max_loa_m` (Float), `max_beam_m` (Float), `max_draft_m` (Float), `max_dwt_mt` (Integer), `has_lightering` (Boolean), `lightering_note` (Text), `source` (String), `is_active` (Boolean, default True) → (Om — Backend Developer)

Task 65: Create file `backend/app/models/reference_vessel_class.py` — define SQLAlchemy `ReferenceVesselClass` model with columns: `id` (Integer PK), `class_name` (String, unique), `dwt_min` (Integer), `dwt_max` (Integer), `typical_draft_m` (Float), `typical_loa_m` (Float), `typical_beam_m` (Float), `avg_speed_knots` (Float) → (Om — Backend Developer)

Task 66: Create file `backend/app/models/reference_cargo_type.py` — define SQLAlchemy `ReferenceCargoType` model with columns: `id` (Integer PK), `cargo_name` (String, unique), `density_mt_per_cbm` (Float), `is_active` (Boolean, default True) → (Om — Backend Developer)

Task 67: Create file `backend/app/models/reference_plant.py` — define SQLAlchemy `ReferencePlant` model with columns: `id` (Integer PK), `plant_name` (String, unique), `location_city` (String), `is_active` (Boolean, default True) → (Om — Backend Developer)

Task 68: Create file `backend/app/models/analysis.py` — define SQLAlchemy `Analysis` model with columns: `id` (UUID PK), `user_id` (UUID FK → users.id), `plant_id` (Integer FK → reference_plants.id, nullable), `cargo_type_id` (Integer FK →

reference_cargo_types.id), quantity_mt (Float), origin_port (String), destination_port_id (Integer FK → reference_ports.id), delivery_start (Date), delivery_end (Date), status (String: DRAFT/PROCESSING/COMPLETE/OVERRIDDEN), current_stock_mt (Float, nullable), daily_consumption_mt (Float, nullable), created_at (DateTime, server_default now), updated_at (DateTime, onupdate now) → **(Om — Backend Developer)**

Task 69: Create file `backend/app/models/context_object.py` — define SQLAlchemy `ContextObject` model with columns: `id` (UUID PK), `analysis_id` (UUID FK → analyses.id, unique), `route_distance_nm` (Float, nullable), `inferred_vessel_class` (String, nullable), `origin_lat` (Float, nullable), `origin_lon` (Float, nullable), `destination_lat` (Float, nullable), `destination_lon` (Float, nullable), `context_resolved_at` (DateTime) → **(Om — Backend Developer)**

Task 70: Create file `backend/app/models/enrichment_cache.py` — define SQLAlchemy `EnrichmentCache` model with columns: `id` (Integer PK), `analysis_id` (UUID FK), `data_key` (String), `data_value` (Text), `data_classification` (String: REAL_DATA/VERIFIED_EXTERNAL/DERIVED), `source_url` (String, nullable), `fetches_at` (DateTime), `is_stale` (Boolean, default False) → **(Om — Backend Developer)**

Task 71: Create file `backend/app/models/forecast_result.py` — define SQLAlchemy `ForecastResult` model with columns: `id` (UUID PK), `analysis_id` (UUID FK, unique), `p10_usd_per_mt` (Float), `p50_usd_per_mt` (Float), `p90_usd_per_mt` (Float), `arima_baseline_usd_per_mt` (Float), `confidence_label` (String: LOW/MEDIUM/HIGH), `model_used` (String), `forecast_generated_at` (DateTime) → **(Om — Backend Developer)**

Task 72: Create file `backend/app/models/feasibility_result.py` — define SQLAlchemy `FeasibilityResult` model with columns: `id` (UUID PK), `analysis_id` (UUID FK), `vessel_class` (String), `port_id` (Integer FK), `draft_pass` (Boolean), `loa_pass` (Boolean), `beam_pass` (Boolean), `dwt_pass` (Boolean), `overall_feasible` (Boolean), `requires_lightering` (Boolean), `failure_reason` (Text, nullable) → **(Om — Backend Developer)**

Task 73: Create file `backend/app/models/landed_cost.py` — define SQLAlchemy `LandedCost` model with columns: `id` (UUID PK), `analysis_id` (UUID FK, unique), `freight_rate_usd_per_mt` (Float), `baf_surcharge_usd_per_mt` (Float), `usd_inr_rate` (Float), `total_usd_per_mt` (Float), `total_inr_per_mt` (Float), `total_inr` (Float), `computed_at` (DateTime) → **(Om — Backend Developer)**

Task 74: Create file `backend/app/models/risk_result.py` — define SQLAlchemy `RiskResult` model with columns: `id` (UUID PK), `analysis_id` (UUID FK), `risk_category` (String), `severity` (String: LOW/MEDIUM/HIGH/NOT_ASSESSED), `signal_description` (Text, nullable), `data_source` (String, nullable) → **(Om — Backend Developer)**

Task 75: Create file `backend/app/models/recommendation.py` — define SQLAlchemy

Recommendation model with columns: `id` (UUID PK), `analysis_id` (UUID FK), `rank` (Integer), `vessel_class` (String), `port_id` (Integer FK), `cost_score` (Float), `confidence_score` (Float), `coverage_fit_score` (Float), `total_score` (Float), `cost_score_breakdown` (Text, JSON string), `is_emergency_mode` (Boolean, default False) → **(Om — Backend Developer)**

Task 76: Create file `backend/app/models/decision_record.py` — define SQLAlchemy `DecisionRecord` model with columns: `id` (UUID PK), `analysis_id` (UUID FK, unique), `chosen_vessel_class` (String), `chosen_port_id` (Integer FK), `was_override` (Boolean), `override_reason` (Text, nullable), `decided_by` (UUID FK → `users.id`), `decided_at` (DateTime) → **(Om — Backend Developer)**

Task 77: Create file `backend/app/models/regret_score.py` — define SQLAlchemy `RegretScore` model with columns: `id` (UUID PK), `decision_record_id` (UUID FK, unique), `regret_pct` (Float), `chosen_day_rate` (Float), `best_rate_in_window` (Float), `window_start` (Date), `window_end` (Date), `computed_at` (DateTime) → **(Om — Backend Developer)**

Task 78: Create file `backend/app/models/disruption_alert.py` — define SQLAlchemy `DisruptionAlert` model with columns: `id` (Integer PK autoincrement), `keyword_matched` (String), `headline_text` (Text), `source_url` (String), `matched_at` (DateTime), `is_active` (Boolean, default True) → **(Om — Backend Developer)**

Task 79: Create file `backend/app/models/audit_log.py` — define SQLAlchemy `AuditLog` model with columns: `id` (Integer PK autoincrement), `user_id` (UUID FK, nullable), `action_type` (String), `affected_record_id` (String, nullable), `ip_address` (String, nullable), `logged_at` (DateTime, server_default now), `detail` (Text, nullable) → **(Om — Backend Developer)**

Task 80: Edit `backend/app/models/__init__.py` — import all 17 models so SQLAlchemy metadata picks them up → **(Om — Backend Developer)**

✓ VERIFICATION CHECKPOINT

☐ Run `cd backend && python -c "from app.models import *; print('All models imported')"` — no errors

Alembic Migration

Task 81: Edit `backend/alembic/env.py` — import `Base` from `app.database` and set `target_metadata = Base.metadata` → **(Param — DevOps + Backend)**

Task 82: Edit `backend/alembic.ini` — set `sqlalchemy.url` to match `DATABASE_URL` value → **(Param — DevOps + Backend)**

Task 83: Run `cd backend && alembic revision --autogenerate -m "create_all_17_tables"` — generates a migration file in `alembic/versions/` →

(Param — DevOps + Backend)

Task 84: Open the generated migration file — verify it contains `CREATE TABLE` statements for all 17 tables — do NOT run yet until verified → (Param — DevOps + Backend)

Task 85: Run `cd backend && alembic upgrade head` — applies migration to `astitva_dev` database → (Param — DevOps + Backend)

✓ VERIFICATION CHECKPOINT

- ☐ Run `psql astitva_dev -c "\dt"` — shows 17 tables
- ☐ Run `psql astitva_dev -c "\d analyses"` — shows all columns for analyses table
- ☐ No alembic errors

Reference Data Seed

Task 86: Edit `scripts/seed_reference_data.py` — add function `seed_ports()` that inserts exactly 4 port records with VERIFIED values:

- Paradip: `max_loa_m=300, max_beam_m=46, max_draft_m=16.5, max_dwt_mt=155000, has_lightering=False`
- Dhamra: `max_loa_m=290, max_beam_m=47, max_draft_m=18.0, max_dwt_mt=180000, has_lightering=False`
- Gangavaram: `max_loa_m=300, max_beam_m=50, max_draft_m=21.0, max_dwt_mt=200000, has_lightering=False`
- Haldia: `max_loa_m=240, max_beam_m=32.26, max_draft_m=9.1, max_dwt_mt=50000, has_lightering=True, lightering_note="Lightering via Sagar-Sandheads anchorages"`

→ (Ashish — Backend Lead)

Task 87: Edit `scripts/seed_reference_data.py` — add function `seed_vessel_classes()` that inserts 4 vessel class records:

- Handysize: `dwt_min=25000, dwt_max=40000, typical_draft_m=9.5, typical_loa_m=180, typical_beam_m=28, avg_speed_knots=13`
- Supramax: `dwt_min=50000, dwt_max=65000, typical_draft_m=12.8, typical_loa_m=200, typical_beam_m=32, avg_speed_knots=13.5`
- Panamax: `dwt_min=65000, dwt_max=90000, typical_draft_m=14.2, typical_loa_m=225, typical_beam_m=32.2, avg_speed_knots=13`
- Capesize: `dwt_min=100000, dwt_max=200000, typical_draft_m=18.2, typical_loa_m=292, typical_beam_m=45, avg_speed_knots=14.5`

→ (Ashish — Backend Lead)

Task 88: Edit `scripts/seed_reference_data.py` — add function `seed_cargo_types()` that inserts 2 cargo records: `coking_coal (density=0.85)`, `thermal_coal (density=0.75)`

→ (Om — Backend Developer)

Task 89: Edit `scripts/seed_reference_data.py` — add function `seed_plants()` that inserts 5 plant records: Bhilai, Rourkela, Durgapur, Bokaro, Burnpur → (Om — Backend Developer)

Task 90: Edit `scripts/seed_reference_data.py` — add `if __name__ == "__main__":` block that calls all 4 seed functions in order → (Om — Backend Developer)

Task 91: Run `cd backend && python ../scripts/seed_reference_data.py` → (Om — Backend Developer)

✓ VERIFICATION CHECKPOINT

- ☐ Run `psql astitva_dev -c "SELECT port_name, max_draft_m FROM reference_ports;"` — shows 4 ports with correct draft values
- ☐ Run `psql astitva_dev -c "SELECT class_name, dwt_min, dwt_max FROM reference_vessel_classes;"` — shows 4 vessel classes
- ☐ Haldia record shows `has_lightering = true`

🚀 GIT CHECKPOINT — Commit & Push

```
git add .
git commit -m "feat(db): all 17 ORM models, alembic migration, and reference data"
git push origin feat/foundation
```

🧠 PHASE 1B: BACKEND CORE — AUTH + ENRICHMENT CONNECTORS (Hours 4-16) — Ashish + Om

Goal: JWT auth, all 8 data connectors, context resolution engine, Pydantic schemas.

Ashish owns auth and routers. Om owns schemas, connectors, and engines.

Both work on `feat/backend-core`.

Task 92: Run `git checkout main && git pull origin main && git checkout -b feat/backend-core` → (Ashish — PM + Backend Lead)

Pydantic Schemas

Task 93: Create file `backend/app/models/schemas.py` — define `UserCreate` schema with fields: `email` (`EmailStr`), `password` (`str`, `min_length=8`), `role` (`str`) → (Om — Backend Developer)

Task 94: Add to `backend/app/models/schemas.py` — define `UserResponse` schema with fields: `id` (`UUID`), `email` (`str`), `role` (`str`), `is_active` (`bool`) — configure with

`from_attributes = True` → (Om — Backend Developer)

Task 95: Add to `backend/app/models/schemas.py` — define `TokenResponse` schema with fields: `access_token` (str), `token_type` (str, default="bearer") → (Om — Backend Developer)

Task 96: Add to `backend/app/models/schemas.py` — define `AnalysisCreate` schema with exactly 5+2 fields: `cargo_type_id` (int), `quantity_mt` (float, gt=0), `origin_port` (str), `destination_port_id` (int), `delivery_start` (date), `delivery_end` (date), `plant_id` (int, optional), `current_stock_mt` (float, optional, gt=0), `daily_consumption_mt` (float, optional, gt=0) — add validator ensuring `delivery_end > delivery_start` → (Om — Backend Developer)

Task 97: Add to `backend/app/models/schemas.py` — define `ContextObjectSchema` with fields: `route_distance_nm` (float, nullable), `inferred_vessel_class` (str, nullable), `origin_lat` (float, nullable), `origin_lon` (float, nullable), `destination_lat` (float, nullable), `destination_lon` (float, nullable) → (Om — Backend Developer)

Task 98: Add to `backend/app/models/schemas.py` — define `EnrichmentDataSchema` with fields: `bunker_price_usd_per_mt` (float, nullable), `bunker_price_data_classification` (str), `bdry_proxy_value` (float, nullable), `bdry_data_classification` (str), `usd_inr_rate` (float, nullable), `forex_data_classification` (str), `weather_flag` (str, nullable), `weather_data_classification` (str), `fetch_timestamp` (datetime, nullable) → (Om — Backend Developer)

Task 99: Add to `backend/app/models/schemas.py` — define `ForecastResultSchema` with fields: `p10_usd_per_mt` (float), `p50_usd_per_mt` (float), `p90_usd_per_mt` (float), `arima_baseline_usd_per_mt` (float), `confidence_label` (str), `model_used` (str) → (Om — Backend Developer)

Task 100: Add to `backend/app/models/schemas.py` — define `FeasibilityResultSchema` with fields: `vessel_class` (str), `port_name` (str), `draft_pass` (bool), `loa_pass` (bool), `beam_pass` (bool), `dwt_pass` (bool), `overall_feasible` (bool), `requires_lightering` (bool), `failure_reason` (str, nullable) → (Om — Backend Developer)

Task 101: Add to `backend/app/models/schemas.py` — define `LandedCostSchema` with fields: `freight_rate_usd_per_mt` (float), `bafe_surcharge_usd_per_mt` (float), `usd_inr_rate` (float), `total_usd_per_mt` (float), `total_inr_per_mt` (float), `total_inr` (float) → (Om — Backend Developer)

Task 102: Add to `backend/app/models/schemas.py` — define `RiskResultSchema` with fields: `risk_category` (str), `severity` (str), `signal_description` (str, nullable), `data_source` (str, nullable) → (Om — Backend Developer)

Task 103: Add to `backend/app/models/schemas.py` — define `ScoreBreakdown` schema with fields: `label` (str), `weight` (float), `raw_value` (float), `data_source` (str) → (Om

— Backend Developer)

Task 104: Add to `backend/app/models/schemas.py` — define `RecommendationSchema` with fields: `rank` (int), `vessel_class` (str), `port_name` (str), `cost_score` (float), `confidence_score` (float), `coverage_fit_score` (float), `total_score` (float), `score_breakdown` (List[ScoreBreakdown]), `is_emergency_mode` (bool) → (Om — Backend Developer)

Task 105: Add to `backend/app/models/schemas.py` — define `StockOutAlertSchema` with fields: `days_to_stockout` (float), `days_to_best_window` (float), `is_at_risk` (bool), `alert_message` (str) → (Om — Backend Developer)

Task 106: Add to `backend/app/models/schemas.py` — define `FullAnalysisResponse` schema composing all above schemas into one response object → (Om — Backend Developer)

Task 107: Add to `backend/app/models/schemas.py` — define `DecisionCreate` schema with fields: `chosen_vessel_class` (str), `chosen_port_id` (int), `was_override` (bool), `override_reason` (str, optional) → (Om — Backend Developer)

✓ VERIFICATION CHECKPOINT

☐ Run `cd backend && python -c "from app.models.schemas import FullAnalysisResponse; print('OK')"` — no errors

JWT Authentication

Task 108: Create file `backend/app/utils/auth.py` — define function `hash_password(password: str) -> str` using `passlib bcrypt` → (Ashish — PM + Backend Lead)

Task 109: Add to `backend/app/utils/auth.py` — define function `verify_password(plain: str, hashed: str) -> bool` using `passlib bcrypt verify` → (Ashish — PM + Backend Lead)

Task 110: Add to `backend/app/utils/auth.py` — define function `create_access_token(data: dict) -> str` using `python-jose` to sign a JWT with `JWT_SECRET_KEY`, expiry from `JWT_EXPIRY_MINUTES` → (Ashish — PM + Backend Lead)

Task 111: Add to `backend/app/utils/auth.py` — define function `decode_token(token: str) -> dict` that verifies and decodes the JWT, raising `HTTPException 401` on invalid/expired → (Ashish — PM + Backend Lead)

Task 112: Add to `backend/app/utils/auth.py` — define FastAPI dependency function `get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)) -> User` that calls `decode_token`, queries the users table, and returns the User ORM object → (Ashish — PM + Backend Lead)

Task 113: Create file `backend/app/routers/auth.py` — define `APIRouter` with prefix `/auth`, add `POST /login` endpoint that accepts `email` and `password`, queries the `users` table, verifies password, and returns a `TokenResponse` → **(Ashish — PM + Backend Lead)**

Task 114: Create file `backend/app/services/user_service.py` — define function `create_initial_admin(db, email, password)` that creates the first admin user with a hashed password — used only by a setup script, not exposed as an API endpoint → **(Ashish — PM + Backend Lead)**

Task 115: Create file `scripts/create_admin.py` — a script that calls `create_initial_admin` with hardcoded demo credentials: `admin@astitva.gov.in / AstitvaDemo2026!` → **(Ashish — PM + Backend Lead)**

Task 116: Run `cd backend && python ../scripts/create_admin.py` → **(Ashish — PM + Backend Lead)**

Task 117: Register the auth router in `backend/app/main.py`: add `app.include_router(auth_router, prefix="/auth", tags=["auth"])` → **(Ashish — PM + Backend Lead)**

✓ VERIFICATION CHECKPOINT

- ☐ POST to `http://localhost:8000/auth/login` with `{"email": "admin@astitva.gov.in", "password": "AstitvaDemo2026!"}` returns a JWT token
- ☐ POST with wrong password returns 401

Data Enrichment Connectors

Task 118: Create file `backend/app/connectors/bunker_connector.py` — define async function `fetch_bunker_price(port_name: str) -> dict` that makes an HTTP GET to `SHIP_AND_BUNKER_BASE_URL` and returns `{"price_usd_per_mt": float, "grade": str, "port": str, "fetched_at": datetime, "classification": "VERIFIED_EXTERNAL_DATA"}` — if request fails, return `{"price_usd_per_mt": None, "classification": "UNAVAILABLE", "error": str}` → **(Om — Backend Developer)**

Task 119: Create file `backend/app/connectors/freight_index_connector.py` — define async function `fetch_bdry_proxy() -> dict` that fetches BDRY ETF data from `BDRY_DATA_SOURCE_URL` (Yahoo Finance CSV endpoint for BDRY) and returns latest close price and trailing-30-day values — if fetch fails, return unavailable marker — classification: `VERIFIED_EXTERNAL_DATA` → **(Om — Backend Developer)**

Task 120: Create file `backend/app/connectors/forex_connector.py` — define async function `fetch_usd_inr_rate() -> dict` that fetches the current USD/INR rate from `RBI_FOREX_FEED_URL` and returns `{"usd_inr": float, "fetched_at": datetime, "classification": "VERIFIED_EXTERNAL_DATA"}` → **(Om — Backend Developer)**

Task 121: Create file `backend/app/connectors/weather_connector.py` — define async function `fetch_weather_flag(lat: float, lon: float) -> dict` that calls `WEATHER_API_BASE_URL` (Open-Meteo free endpoint) for the given coordinates and returns a single string flag: "FAVORABLE", "CAUTION", or "ADVERSE" based on wind speed thresholds — classification: `VERIFIED_EXTERNAL_DATA` → **(Om — Backend Developer)**

Task 122: Create file `backend/app/connectors/locode_connector.py` — define function `get_port_coordinates(locode: str) -> dict` that looks up lat/lon from a locally stored UN/LOCODE JSON file (no external API call needed at runtime) and returns `{"lat": float, "lon": float}` or `None` if not found → **(Om — Backend Developer)**

Task 123: Create file `backend/app/connectors/locode_data.json` — a static JSON file containing lat/lon for at minimum these 6 ports: Newcastle (AU), Port Kembla (AU), Baltimore (US), Hampton Roads (US), Paradip (IN), Dhamra (IN), Gangavaram (IN), Haldia (IN) — data sourced from UN/LOCODE registry → **(Om — Backend Developer)**

Task 124: Create file `backend/app/connectors/disruption_connector.py` — define function `fetch_and_scan_disruptions(keyword_list: list[str]) -> list[dict]` that uses `feedparser` to fetch RSS feeds from Journal of Commerce (<https://www.joc.com/rss/all>) and Maritime Executive (<https://maritime-executive.com/feed>) and returns any headlines containing any keyword from `keyword_list` → **(Om — Backend Developer)**

Task 125: Create file `backend/app/connectors/world_bank_connector.py` — define async function `fetch_commodity_price() -> dict` that downloads the World Bank Pink Sheet CSV from the public URL, parses it with `pandas`, and extracts the latest coal price — classification: `VERIFIED_EXTERNAL_DATA` → **(Om — Backend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Run `cd backend && python -c "import asyncio; from app.connectors.forex_connector import fetch_usd_inr_rate; print(asyncio.run(fetch_usd_inr_rate()))"` — returns a dict with a `usd_inr` float value
- ☐ If any connector fails (test by temporarily breaking the URL) it returns an `UNAVAILABLE` marker, not an exception

Context Resolution Engine

Task 126: Create file `backend/app/engines/context_engine.py` — define function `resolve_context(analysis: Analysis, db: Session) -> ContextObject` that executes the locked resolution order: Fetch → Calculate → Derive → Infer → Ask → **(Om — Backend Developer)**

Task 127: Add to `backend/app/engines/context_engine.py` — implement the Fetch

step: call `locode_connector.get_port_coordinates` for both origin and destination, store results in context — if coordinates not found, mark as `unavailable` → **(Om — Backend Developer)**

Task 128: Add to `backend/app/engines/context_engine.py` — implement the Derive step: if both coordinates are available, compute `route_distance_nm` using the haversine formula (great-circle distance approximation, explicitly labeled as DERIVED/ESTIMATE, not actual sailing route) → **(Om — Backend Developer)**

Task 129: Add to `backend/app/engines/context_engine.py` — implement the Infer step: look up the 4 vessel classes from the database, find the class whose `dwt_min` $\leq$ `quantity_mt` $\leq$ `dwt_max`, set `inferred_vessel_class` — if quantity falls between classes, pick the smaller class and note it → **(Om — Backend Developer)**

Task 130: Add to `backend/app/engines/context_engine.py` — implement the enrichment fetch step: call all 4 active connectors (bunker, forex, BDRY, weather) concurrently using `asyncio.gather`, write results to `enrichment_cache` table with `fetched_at` timestamp → **(Om — Backend Developer)**

Task 131: Add to `backend/app/engines/context_engine.py` — implement the stale-cache fallback: if any connector returns UNAVAILABLE, check `enrichment_cache` for a record within the last 24 hours — if found, use cached value and set `is_stale` = `True` on that cache record → **(Om — Backend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Create a test analysis record in the DB manually and call `resolve_context` — verify a `ContextObject` is created with `route_distance_nm` populated
- ☐ Verify `inferred_vessel_class` is correctly set for `quantity_mt=75000` (should be Panamax)

Analysis Router

Task 132: Create file `backend/app/routers/analyses.py` — define `APIRouter` with prefix `/analyses`, protected by `get_current_user` dependency → **(Ashish — PM + Backend Lead)**

Task 133: Add to `backend/app/routers/analyses.py` — implement `GET /analyses` endpoint returning paginated list of analyses for the current user (or all analyses for ADMIN/AUDITOR roles), ordered by `created_at` DESC, default `limit=10` → **(Ashish — PM + Backend Lead)**

Task 134: Add to `backend/app/routers/analyses.py` — implement `POST /analyses` endpoint that: validates `AnalysisCreate` schema, creates an `Analysis` record with status PROCESSING, calls `resolve_context()`, calls the Forecast Engine, Feasibility Engine, Landed Cost Calculator, Stock-Out Alert, Risk Engine, and Recommendation

Engine in sequence, saves all results, updates status to COMPLETE, and returns

`FullAnalysisResponse` → **(Ashish — PM + Backend Lead)**

Task 135: Add to `backend/app/routers/analyses.py` — implement `GET /analyses/{id}` endpoint returning the full `FullAnalysisResponse` for a single analysis by UUID → **(Ashish — PM + Backend Lead)**

Task 136: Add to `backend/app/routers/analyses.py` — implement `POST /analyses/{id}/decision` endpoint accepting `DecisionCreate` schema, creating a `DecisionRecord`, writing an audit log entry, and returning the saved decision → **(Ashish — PM + Backend Lead)**

Task 137: Add to `backend/app/routers/analyses.py` — implement `GET /analyses/{id}/export` endpoint that generates a plain-text structured report (markdown format) containing the full analysis chain — no PDF library needed for MVP, plain text download is sufficient → **(Ashish — PM + Backend Lead)**

Task 138: Register the analyses router in `backend/app/main.py` : add `app.include_router(analyses_router, prefix="/analyses", tags=["analyses"])` → **(Ashish — PM + Backend Lead)**

✓ VERIFICATION CHECKPOINT

- ☐ POST to `/analyses` with valid payload — returns 200 with a full analysis response
- ☐ GET to `/analyses/{id}` — returns the same analysis
- ☐ GET to `/analyses` — returns paginated list

Admin + Audit Log Routers

Task 139: Create file `backend/app/routers/admin.py` — define `APIRouter` with prefix `/admin`, add role-check dependency that raises 403 if current user is not ADMIN → **(Ashish — PM + Backend Lead)**

Task 140: Add to `backend/app/routers/admin.py` — implement `GET /admin/reference/ports` returning all active ports → **(Ashish — PM + Backend Lead)**

Task 141: Add to `backend/app/routers/admin.py` — implement `PUT /admin/reference/ports/{id}` that updates a port's constraint values and requires a `source` field to be non-empty → **(Ashish — PM + Backend Lead)**

Task 142: Add to `backend/app/routers/admin.py` — implement `GET /admin/users` returning all users (ADMIN only) → **(Ashish — PM + Backend Lead)**

Task 143: Add to `backend/app/routers/admin.py` — implement `POST /admin/users` accepting `userCreate`, hashing the password, inserting a new user → **(Ashish — PM + Backend Lead)**

Task 144: Add to `backend/app/routers/admin.py` — implement `PATCH /admin/users/{id}/deactivate` that sets `is_active=False` (soft-delete, never hard-delete) → **(Ashish — PM + Backend Lead)**

Task 145: Create file `backend/app/routers/audit.py` — implement `GET /audit-logs` endpoint returning paginated audit log entries, filterable by `user_id`, `action_type`, and `date_range` query params → **(Ashish — PM + Backend Lead)**

Task 146: Register both routers in `backend/app/main.py` → **(Ashish — PM + Backend Lead)**

APScheduler Jobs

Task 147: Create file `backend/app/services/scheduler.py` — define function `setup_scheduler(app: FastAPI)` that creates an `APScheduler BackgroundScheduler` and adds two jobs: (1) disruption scanner every 30 minutes, (2) BDRY proxy cache refresh every 60 minutes → **(Om — Backend Developer)**

Task 148: Add to `backend/app/services/scheduler.py` — define job function `run_disruption_scanner()` that calls `disruption_connector.fetch_and_scan_disruptions` with the fixed keyword list `["Red Sea", "cyclone", "port strike", "canal blocked", "typhoon", "Hormuz", "Suez", "Mozambique"]` and upserts results into `disruption_alerts` table → **(Om — Backend Developer)**

Task 149: Add `on_startup` event handler in `backend/app/main.py` that calls `setup_scheduler(app)` to start the scheduler when the server boots → **(Om — Backend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Start the backend server — no scheduler errors in the terminal
- ☐ Wait 30 seconds — check `disruption_alerts` table — if any headlines matched keywords, rows exist

🚀 GIT CHECKPOINT — Commit & Push

```
git add .
git commit -m "feat(backend): auth, schemas, connectors, engines, all routers"
git push origin feat/backend-core
```

🔗 PR CHECKPOINT — Open Pull Request

- **Branch:** `feat/backend-core` → `main`
- **PR Title:** `feat(Phase 1B): Backend Core — Auth + Connectors + Engines + Routers — Ashish`
- **Before merging, verify:**

- ☐ Task 117 verified: POST /auth/login returns JWT ✓
- ☐ Task 134 verified: POST /analyses returns FullAnalysisResponse ✓
- ☐ Task 149 verified: Scheduler starts without errors ✓

• Run: `git checkout main && git pull origin main`

MERGE PROCEDURE

```
git checkout main && git pull origin main
git merge --squash feat/backend-core
git commit -m "feat(backend): complete backend core – auth, enrichment, ..."
git push origin main
```

LIKELY CONFLICT FILES:

- `backend/app/main.py` — Palak may have added routes during frontend work
- `backend/requirements.txt` — Palak may have added numpy or other ML packages

 **CONFLICT RESOLUTION:**  Keep ALL `include_router` lines. Keep ALL packages in `requirements.txt`.

PHASE 1C: ML ENGINE (Hours 6–18) — Palak — ML Engineer

Goal: Forecast engine (LightGBM + ARIMA), feasibility engine, recommendation engine, risk engine, landed cost calculator, stock-out alert, regret score, pooling calculator.

Palak works solo on `feat/ml-engine` in parallel with Phase 1B. This is the most specialized work — no one else touches this branch.

Task 150: Run `git checkout main && git pull origin main && git checkout -b feat/ml-engine` → **(Palak — ML Engineer)**

ML Data Pipeline

Task 151: Create file `ml/data/download_bdry_history.py` — a script that downloads 2 years of BDRY daily close prices from Yahoo Finance CSV endpoint and saves to `ml/data/bdry_history.csv` → **(Palak — ML Engineer)**

Task 152: Run `cd ml && python data/download_bdry_history.py` — verify `ml/data/bdry_history.csv` exists and contains at least 400 rows → **(Palak — ML Engineer)**

Task 153: Create file `ml/feature_engineering.py` — define function `build_features(df: pd.DataFrame) -> pd.DataFrame` that adds these columns to a

BDRY time series dataframe: `lag_7` (7-day lag of close price), `lag_14`, `lag_30`, `rolling_mean_14` (14-day rolling mean), `rolling_std_14`, `rolling_mean_30`, `month` (integer month), `quarter` (integer quarter) → **(Palak — ML Engineer)**

Task 154: Add to `ml/feature_engineering.py` — define function `apply_walk_forward_split(df: pd.DataFrame, test_size: int = 60) -> tuple` that splits the dataframe into train and test sets in chronological order (`shuffle=False` is enforced — add an assertion to verify the test set's first date is after the train set's last date) → **(Palak — ML Engineer)**

✓ VERIFICATION CHECKPOINT

- ☐ Run `python -c "from ml.feature_engineering import build_features; print('OK')"` — no errors
- ☐ Verify the walk-forward split assertion would catch shuffled data (add a test with shuffled data)

LightGBM Quantile Forecast Model

Task 155: Create file `ml/train_lgbm.py` — define function `train_quantile_models(df: pd.DataFrame) -> dict` that trains THREE separate LightGBM regressors with `objective="quantile"` and `alpha` values of 0.10, 0.50, 0.90 respectively, using the features from Task 153 — returns a dict `{"p10": model, "p50": model, "p90": model}` → **(Palak — ML Engineer)**

Task 156: Add to `ml/train_lgbm.py` — define function `evaluate_models(models: dict, X_test, y_test) -> dict` that computes MAE for the p50 model on the test set and prints it clearly — this is the validation metric shown in the demo → **(Palak — ML Engineer)**

Task 157: Add to `ml/train_lgbm.py` — define function `save_models(models: dict, path: str)` that saves all three LightGBM models to `ml/models/lgbm_p10.pkl`, `ml/models/lgbm_p50.pkl`, `ml/models/lgbm_p90.pkl` using `joblib` → **(Palak — ML Engineer)**

Task 158: Add `if __name__ == "__main__":` block to `ml/train_lgbm.py` that loads `bdry_history.csv`, builds features, splits, trains, evaluates, and saves → **(Palak — ML Engineer)**

Task 159: Run `cd ml && python train_lgbm.py` — verify 3 model files are created in `ml/models/` → **(Palak — ML Engineer)**

✓ VERIFICATION CHECKPOINT

- ☐ `ml/models/lgbm_p10.pkl`, `lgbm_p50.pkl`, `lgbm_p90.pkl` all exist
- ☐ MAE for p50 model is printed to terminal (any number — we only need it to run, not a specific accuracy)

ARIMA Baseline Model

Task 160: Create file `ml/train_arima.py` — define function

`fit_arima_baseline(series: pd.Series) -> dict` that fits a `ARIMA(5,1,0)` model using `statsmodels` `ARIMA` class on the training series and returns the fitted model result → **(Palak — ML Engineer)**

Task 161: Add to `ml/train_arima.py` — define function

`arima_forecast(model_result, steps: int = 14) -> float` that generates a `steps` - day forecast and returns the mean of the forecast as a single baseline value → **(Palak — ML Engineer)**

Forecast Engine Service

Task 162: Create file `backend/app/engines/forecast_engine.py` — define function

`run_forecast(analysis: Analysis, enrichment_data: dict) -> ForecastResult` that loads the 3 saved LightGBM models from `MODEL_REGISTRY_PATH`, builds a feature vector from `enrichment_data`, generates p10/p50/p90 predictions, runs the ARIMA baseline, and returns a `ForecastResult` ORM object → **(Palak — ML Engineer)**

Task 163: Add to `backend/app/engines/forecast_engine.py` — define function

`compute_confidence_label(p10: float, p50: float, p90: float) -> str` that returns "HIGH" if $(p90-p10)/p50 < 0.15$, "MEDIUM" if between 0.15 and 0.30, "LOW" if above 0.30 → **(Palak — ML Engineer)**

Task 164: Add to `backend/app/engines/forecast_engine.py` — add a fallback: if models fail to load (e.g., on first run before training), return a `ForecastResult` with all values set to None and `confidence_label = "UNAVAILABLE"` — never raise an unhandled exception → **(Palak — ML Engineer)**

✓ VERIFICATION CHECKPOINT

- ☐ Run `python -c "from app.engines.forecast_engine import run_forecast; print('OK')"` — no import errors
 - ☐ Verify that calling `run_forecast` with mock enrichment data returns a `ForecastResult` object with non-None p50
-

Feasibility Engine

Task 165: Create file `backend/app/engines/feasibility_engine.py` — define function

`run_feasibility_check(analysis: Analysis, db: Session) -> list[FeasibilityResult]` that queries all 4 vessel classes and the destination port's constraints and runs the feasibility check for each vessel class → **(Palak — ML Engineer)**

Task 166: Add to `backend/app/engines/feasibility_engine.py` — for each vessel class, evaluate: `draft_pass = vessel.typical_draft_m <= port.max_draft_m`, `loa_pass = vessel.typical_loa_m <= port.max_loa_m`, `beam_pass = vessel.typical_beam_m <= port.max_beam_m`, `dwt_pass = vessel.dwt_max >= analysis.quantity_mt` — `overall_feasible = all four pass` → **(Palak — ML Engineer)**

Task 167: Add to `backend/app/engines/feasibility_engine.py` — for Haldia: if `draft_pass` is False but `port.has_lightering` is True, set `overall_feasible = True` and `requires_lightering = True` — this is the Haldia special case → **(Palak — ML Engineer)**

Task 168: Add to `backend/app/engines/feasibility_engine.py` — set `failure_reason` to a human-readable string for any failed check (e.g., "Draft 18.2m exceeds Haldia max draft 9.1m") → **(Palak — ML Engineer)**

Recommendation Engine

Task 169: Create file `backend/app/engines/recommendation_engine.py` — define function `run_recommendation(analysis: Analysis, forecast: ForecastResult, feasibility_results: list[FeasibilityResult], is_emergency: bool = False) -> list[Recommendation]` → **(Palak — ML Engineer)**

Task 170: Add to `backend/app/engines/recommendation_engine.py` — filter `feasibility_results` to only `overall_feasible = True` entries — if `is_emergency = True`, exclude any "wait" option (i.e., ensure at least one option is always returned) → **(Palak — ML Engineer)**

Task 171: Add to `backend/app/engines/recommendation_engine.py` — compute `cost_score` for each feasible option: normalize `p50_usd_per_mt` across feasible options to [0,1] scale, invert (lower cost = higher score) → **(Palak — ML Engineer)**

Task 172: Add to `backend/app/engines/recommendation_engine.py` — compute `confidence_score` from `forecast.confidence_label`: HIGH=1.0, MEDIUM=0.6, LOW=0.3 → **(Palak — ML Engineer)**

Task 173: Add to `backend/app/engines/recommendation_engine.py` — compute `coverage_fit_score`: ratio of `analysis.quantity_mt` to `vessel_class.dwt_max`, capped at 1.0 (higher fill = better fit) → **(Palak — ML Engineer)**

Task 174: Add to `backend/app/engines/recommendation_engine.py` — compute `total_score = 0.5 * cost_score + 0.3 * confidence_score + 0.2 * coverage_fit_score` (exact locked weights — never change these constants) → **(Palak — ML Engineer)**

Task 175: Add to `backend/app/engines/recommendation_engine.py` — sort results by `total_score DESC` — on ties, the lower p50 cost wins (deterministic tie-breaking) →

(Palak — ML Engineer)

Task 176: Add to `backend/app/engines/recommendation_engine.py` — build `score_breakdown` list for each recommendation containing 3 `ScoreBreakdown` items: one for each sub-score with its label, weight, raw value, and data source string → (Palak — ML Engineer)

Risk Engine

Task 177: Create file `backend/app/engines/risk_engine.py` — define function `run_risk_assessment(analysis: Analysis, enrichment_data: dict, feasibility_results: list) -> list[RiskResult]` that evaluates 6 risk categories and returns a list of `RiskResult` objects → (Palak — ML Engineer)

Task 178: Add to `backend/app/engines/risk_engine.py` — risk 1: freight rate volatility — compute $(p_{90} - p_{10}) / p_{50}$ from forecast — HIGH if >0.30 , MEDIUM if $0.15-0.30$, LOW if <0.15 → (Palak — ML Engineer)

Task 179: Add to `backend/app/engines/risk_engine.py` — risk 2: port draft constraint tightness — for the selected destination, if `vessel.typical_draft_m / port.max_draft_m > 0.90`, severity = HIGH; if >0.75 , MEDIUM; else LOW → (Palak — ML Engineer)

Task 180: Add to `backend/app/engines/risk_engine.py` — risk 3: delivery window tightness — if `delivery_end - delivery_start < 7 days`, HIGH; if <14 days, MEDIUM; else LOW → (Palak — ML Engineer)

Task 181: Add to `backend/app/engines/risk_engine.py` — risk 4: bunker price volatility — compute trailing 30-day standard deviation from BDRY data — HIGH if $\text{stddev} > 20\%$ of mean; MEDIUM if $>10\%$; LOW otherwise → (Palak — ML Engineer)

Task 182: Add to `backend/app/engines/risk_engine.py` — risk 5: vessel availability — always `severity = "NOT_ASSESSED"`, `signal_description = "Individual vessel availability cannot be checked – no free real-time AIS source available"` → (Palak — ML Engineer)

Task 183: Add to `backend/app/engines/risk_engine.py` — risk 6: geopolitical disruption — check `disruption_alerts` table for any `is_active = True` alerts — if any exist, `severity = "LOW"` and `signal_description = "Keyword-matched news signal: {keyword}. Not a validated risk assessment."` — if none, `severity = "NOT_ASSESSED"` → (Palak — ML Engineer)

Landed Cost Calculator

Task 184: Create file `backend/app/engines/landed_cost_engine.py` — define function `calculate_landed_cost(analysis: Analysis, forecast: ForecastResult,`

enrichment_data: dict) -> LandedCost → (Palak — ML Engineer)

Task 185: Add to `backend/app/engines/landed_cost_engine.py` — extract
`freight_rate_usd_per_mt = forecast.p50_usd_per_mt , baf_surcharge_usd_per_mt`
`= enrichment_data.get("bunker_price_usd_per_mt", 0) * 0.05` (5% of bunker price
as BAF approximation, labeled ENGINEERING ASSUMPTION), `usd_inr_rate =`
`enrichment_data.get("usd_inr_rate")` → (Palak — ML Engineer)

Task 186: Add to `backend/app/engines/landed_cost_engine.py` — compute:
`total_usd_per_mt = freight_rate_usd_per_mt + baf_surcharge_usd_per_mt ,`
`total_inr_per_mt = total_usd_per_mt * usd_inr_rate , total_inr =`
`total_inr_per_mt * analysis.quantity_mt` → (Palak — ML Engineer)

Stock-Out Alert Engine

Task 187: Create file `backend/app/engines/stockout_engine.py` — define function
`calculate_stockout_alert(analysis: Analysis, forecast: ForecastResult) ->`
`StockOutAlert | None` → (Palak — ML Engineer)

Task 188: Add to `backend/app/engines/stockout_engine.py` — if
`analysis.current_stock_mt` is None or `analysis.daily_consumption_mt` is None,
return None (user did not provide stock data) → (Palak — ML Engineer)

Task 189: Add to `backend/app/engines/stockout_engine.py` — compute
`days_to_stockout = current_stock_mt / daily_consumption_mt` → (Palak — ML
Engineer)

Task 190: Add to `backend/app/engines/stockout_engine.py` — compute
`days_to_best_window` : find the day within the 14-day forecast window where p10
(best-case rate) is lowest — derive as `best_day_index + 1` from the forecast output
→ (Palak — ML Engineer)

Task 191: Add to `backend/app/engines/stockout_engine.py` — set `is_at_risk =`
`days_to_stockout < days_to_best_window` — build `alert_message` string: "Stock
will last {days_to_stockout:.0f} days. Next favorable rate window is
{days_to_best_window:.0f} days away. {'Book now – cannot afford to wait.' if
`is_at_risk` else 'Safe to wait for a better rate.'}" → (Palak — ML Engineer)

Spot vs. COA Comparison Engine

Task 192: Create file `backend/app/engines/coa_comparison_engine.py` — define
function `calculate_coa_comparison(spot_landed_cost: LandedCost,`
`coa_discount_pct: float, num_voyages: int) -> dict` → (Palak — ML Engineer)

Task 193: Add to `backend/app/engines/coa_comparison_engine.py` — compute:
`coa_rate_per_mt = spot_landed_cost.total_inr_per_mt * (1 - coa_discount_pct`

```
/ 100), spot_total = spot_landed_cost.total_inr * num_voyages, coa_total =  
coa_rate_per_mt * spot_landed_cost.analysis.quantity_mt * num_voyages,  
savings = spot_total - coa_total → (Palak — ML Engineer)
```

Task 194: Add to `backend/app/engines/coa_comparison_engine.py` — hardcode a docstring note: `# coa_discount_pct` is an ENGINEERING ASSUMPTION — default 8%. Admin can override. Never present as market fact. → (Palak — ML Engineer)

Multi-Plant Pooling Engine

Task 195: Create file `backend/app/engines/pooling_engine.py` — define function `calculate_pooling(analysis_a: Analysis, analysis_b: Analysis, db: Session) -> dict` → (Palak — ML Engineer)

Task 196: Add to `backend/app/engines/pooling_engine.py` — validate: both analyses must have the same `destination_port_id` — raise `ValueError` if not → (Palak — ML Engineer)

Task 197: Add to `backend/app/engines/pooling_engine.py` — compute `combined_quantity = analysis_a.quantity_mt + analysis_b.quantity_mt` → (Palak — ML Engineer)

Task 198: Add to `backend/app/engines/pooling_engine.py` — find the feasible vessel class for `combined_quantity` by querying vessel classes where `dwt_max >= combined_quantity` and selecting the smallest (most efficient) — returns `None` if no vessel class is large enough → (Palak — ML Engineer)

Task 199: Add to `backend/app/engines/pooling_engine.py` — compute `per_tonne_saving_pct`: use the p50 forecast rates from both individual analyses and the combined analysis — return a dict with: `combined_quantity`, `combined_vessel_class`, `individual_vessel_class_a`, `individual_vessel_class_b`, `estimated_saving_note` (string, labeled as ENGINEERING ESTIMATE) → (Palak — ML Engineer)

Regret Score Engine

Task 200: Create file `backend/app/engines/regret_engine.py` — define function `compute_regret_score(decision_record_id: UUID, db: Session) -> RegretScore` → (Palak — ML Engineer)

Task 201: Add to `backend/app/engines/regret_engine.py` — load the decision record and the analysis's `delivery_start` date → (Palak — ML Engineer)

Task 202: Add to `backend/app/engines/regret_engine.py` — define a `±14 day` window around `delivery_start` — fetch BDRY close prices for those 28 days from `enrichment_cache` or from the saved `bdry_history.csv` → (Palak — ML Engineer)

Task 203: Add to `backend/app/engines/regret_engine.py` — compute

```
best_rate_in_window = min(prices), chosen_day_rate = price on decision date,  
regret_pct = (chosen_day_rate - best_rate_in_window) / best_rate_in_window *  
100 → (Palak — ML Engineer)
```

Task 204: Add to `backend/app/engines/regret_engine.py` — save and return a

`RegretScore` ORM object — add a `computed_at` timestamp — never run this calculation if `decision_record` is less than 14 days old (no future data) → (Palak — ML Engineer)

✓ VERIFICATION CHECKPOINT

- ☐ All 8 engine files exist in `backend/app/engines/`
- ☐ Run `python -c "from app.engines import forecast_engine, feasibility_engine, recommendation_engine, risk_engine, landed_cost_engine, stockout_engine, coa_comparison_engine, pooling_engine, regret_engine; print('All engines OK')"` — no errors

🚀 GIT CHECKPOINT — Commit & Push

```
git add .  
git commit -m "feat(ml): complete all 9 engines – forecast, feasibility,  
git push origin feat/ml-engine"
```

🔗 PR CHECKPOINT — Open Pull Request

- **Branch:** `feat/ml-engine` → `main`
- **PR Title:** `feat(Phase 1C): Complete ML + Decision Engines – Palak`
- **Before merging, verify:**
 - ☐ Task 159 verified: 3 LightGBM model files exist ✓
 - ☐ Task 174 verified: recommendation score formula uses 0.5/0.3/0.2 exactly ✓
 - ☐ Task 191 verified: stock-out alert returns correct message for at-risk case ✓
- Run: `git checkout main && git pull origin main`

🔗 MERGE PROCEDURE

```
git checkout main && git pull origin main  
git merge --squash feat/ml-engine  
git commit -m "feat(ml): complete all engines and ML pipeline"  
git push origin main
```

⚠️ LIKELY CONFLICT FILES:

- `backend/requirements.txt` — ML packages added by Palak may conflict with Ashish's additions

- `backend/app/models/schemas.py` — both members may have extended this

💡 **CONFLICT RESOLUTION:** 🔴 Keep ALL packages in `requirements.txt`. For `schemas.py`, keep ALL schema classes — never delete one added by the other member.

🎨 PHASE 2A: FRONTEND UI CORE (Hours 18–28) — Prachi + Mahima

Goal: Core authenticated flow — Login, Dashboard, New Analysis, and the full Analysis Results page (11 sections) — connected to the real backend. Prachi owns auth flow and input pages. Mahima owns the results page and map component.

Task 205: Run `git checkout main && git pull origin main && git checkout -b feat/frontend-ui` → **(Prachi — Frontend Developer)**

TypeScript Types (Finalize)

Task 206: Edit `frontend/src/types/index.ts` — fill in the `Analysis` interface with all fields matching the `FullAnalysisResponse` schema from Task 106 → **(Prachi — Frontend Developer)**

Task 207: Edit `frontend/src/types/index.ts` — fill in `ForecastResult`, `FeasibilityResult`, `RiskResult`, `RecommendationResult`, `LandedCost`, `StockOutAlert`, `DisruptionAlert`, `RegretScore` interfaces → **(Prachi — Frontend Developer)**

API Layer

Task 208: Create file `frontend/src/api/auth.ts` — define function `login(email: string, password: string): Promise<TokenResponse>` using the Axios client → **(Prachi — Frontend Developer)**

Task 209: Create file `frontend/src/api/analyses.ts` — define function `createAnalysis(payload: AnalysisCreate): Promise<FullAnalysisResponse>` → **(Prachi — Frontend Developer)**

Task 210: Add to `frontend/src/api/analyses.ts` — define function `getAnalysis(id: string): Promise<FullAnalysisResponse>` → **(Prachi — Frontend Developer)**

Task 211: Add to `frontend/src/api/analyses.ts` — define function `listAnalyses(limit?: number): Promise<Analysis[]>` → **(Prachi — Frontend Developer)**

Task 212: Add to `frontend/src/api/analyses.ts` — define function

`submitDecision(analysisId: string, payload: DecisionCreate):`

`Promise<DecisionRecord>` → **(Prachi — Frontend Developer)**

Task 213: Create file `frontend/src/api/admin.ts` — define functions: `listUsers()`, `createUser(payload)`, `deactivateUser(id)`, `listPorts()`, `updatePort(id, payload)` → **(Prachi — Frontend Developer)**

Task 214: Create file `frontend/src/api/audit.ts` — define function

`listAuditLogs(params?)` → **(Prachi — Frontend Developer)**

Auth Context + Protected Routes

Task 215: Create file `frontend/src/context/AuthContext.tsx` — define a React context with `user`, `token`, `login(email, password)`, `logout()` — store token in `localStorage` on login, clear on logout → **(Prachi — Frontend Developer)**

Task 216: Create file `frontend/src/components/ProtectedRoute.tsx` — a wrapper component that reads the auth context and redirects to `/login` if no token is present → **(Prachi — Frontend Developer)**

Task 217: Edit `frontend/src/App.tsx` — wrap all routes except `/login` with

`<ProtectedRoute>` → **(Prachi — Frontend Developer)**

Shared Layout Component

Task 218: Create file `frontend/src/components/Layout.tsx` — a wrapper component with a top navigation bar containing: Astitva logo text (left), nav links to Dashboard / New Analysis / Audit Log (center), user role display + Logout button (right) → **(Prachi — Frontend Developer)**

Task 219: Edit all non-login page files — wrap their return value with `<Layout>` → **(Prachi — Frontend Developer)**

Login Page

Task 220: Edit `frontend/src/pages/LoginPage.tsx` — add a centered card with an email input, password input, submit button labeled "Sign In", and an error message area → **(Prachi — Frontend Developer)**

Task 221: Add to `LoginPage.tsx` — on form submit, call `authContext.login(email, password)` — on success, navigate to `/dashboard` — on failure, show "Invalid email or password" message → **(Prachi — Frontend Developer)**

- ☐ Navigate to `/login` — form renders with two inputs and a button
- ☐ Submit with correct demo credentials — redirects to `/dashboard`
- ☐ Submit with wrong credentials — shows error message

Dashboard Page

Task 222: Edit `frontend/src/pages/DashboardPage.tsx` — call `listAnalyses(10)` on mount using a `useEffect` — show a loading spinner while fetching → **(Prachi — Frontend Developer)**

Task 223: Add to `DashboardPage.tsx` — render a table with columns: Date | Cargo Type | Origin → Destination | Status | View — each row links to `/analyses/{id}` → **(Prachi — Frontend Developer)**

Task 224: Add to `DashboardPage.tsx` — render a prominent "New Analysis" button linking to `/analyses/new` → **(Prachi — Frontend Developer)**

Task 225: Add to `DashboardPage.tsx` — render a "Disruption Alerts" section below the table — fetch from `GET /disruption-alerts` endpoint — if any active alerts, show each as a red banner with the keyword and headline — if none, show "No active disruption alerts" in green → **(Prachi — Frontend Developer)**

Task 226: Add to `DashboardPage.tsx` — render an empty state card ("No analyses yet. Click New Analysis to start.") when the list is empty → **(Prachi — Frontend Developer)**

New Analysis Page

Task 227: Edit `frontend/src/pages/NewAnalysisPage.tsx` — add a form with the 5 required input fields: cargo type (dropdown, fetched from `GET /reference/cargo-types`), quantity_mt (number input), origin_port (text input with helper text "e.g. Newcastle, AU"), destination_port (dropdown — exactly 4 options: Paradip, Dhamra, Gangavaram, Haldia), delivery_start + delivery_end (date inputs) → **(Prachi — Frontend Developer)**

Task 228: Add to `NewAnalysisPage.tsx` — add a collapsible "Optional: Stock-Out Prevention" section with two extra inputs: current_stock_mt and daily_consumption_mt — collapsed by default, expandable with a toggle → **(Prachi — Frontend Developer)**

Task 229: Add to `NewAnalysisPage.tsx` — add a plant selector dropdown (optional) populated from `GET /reference/plants` — labeled "Your SAIL Plant (optional)" → **(Prachi — Frontend Developer)**

Task 230: Add to `NewAnalysisPage.tsx` — add client-side validation: all 5 required fields must be filled, delivery_end must be after delivery_start, quantity_mt must be >

0 — disable the Submit button until all validations pass → **(Prachi — Frontend Developer)**

Task 231: Add to `NewAnalysisPage.tsx` — on form submit, call `createAnalysis(payload)` — show a full-page loading state with the message "System is automatically resolving context, fetching live data, and running the forecast..." during the API call → **(Prachi — Frontend Developer)**

Task 232: Add to `NewAnalysisPage.tsx` — on success, navigate to `/analyses/{id}` with the returned analysis ID → **(Prachi — Frontend Developer)**

Analysis Results Page

Task 233: Edit `frontend/src/pages/AnalysisResultsPage.tsx` — call `getAnalysis(id)` on mount and store the result in state → **(Mahima — Frontend Developer)**

Task 234: Add to `AnalysisResultsPage.tsx` — Section 1: "Context Summary" — show a small card: Origin → Destination, distance (with note "approximate great-circle distance"), inferred vessel class, data fetch timestamp → **(Mahima — Frontend Developer)**

Task 235: Add to `AnalysisResultsPage.tsx` — Section 2: "Freight Rate Forecast" — render a simple horizontal bar or number display showing P10 / P50 / P90 values in \$/MT, with a label "P50 = most likely rate, P10 = best case, P90 = worst case" — show ARIMA baseline value alongside — show confidence label as a badge (HIGH=green, MEDIUM=yellow, LOW=red) → **(Mahima — Frontend Developer)**

Task 236: Add to `AnalysisResultsPage.tsx` — Section 3: "Total Landed Cost" — render the 4-row breakdown card: Freight Rate + BAF Surcharge = Subtotal × Forex Rate = **Total in ₹/MT** and **Total in ₹** — tag each row with its data classification badge (VERIFIED EXTERNAL / DERIVED / MODEL OUTPUT) → **(Mahima — Frontend Developer)**

Task 237: Add to `AnalysisResultsPage.tsx` — Section 4: "Vessel & Port Feasibility" — render a table with columns: Vessel Class | Draft  | LOA  | Beam  | DWT  | Overall | Notes — one row per vessel class → **(Mahima — Frontend Developer)**

Task 238: Add to `AnalysisResultsPage.tsx` — Section 5: "Stock-Out Alert" — only render this section if `stockout_alert` is not null — show the alert card with `days_to_stockout`, `days_to_best_window`, and the `alert_message` — if `is_at_risk` = `true`, show card with a red border; if false, show with a green border → **(Mahima — Frontend Developer)**

Task 239: Add to `AnalysisResultsPage.tsx` — Section 6: "Risk Assessment" — render a table with columns: Risk Category | Severity | Signal | Data Source — use color-coded severity badges — for NOT_ASSESSED rows, show a grey badge with an info

tooltip explaining why → **(Mahima — Frontend Developer)**

Task 240: Add to `AnalysisResultsPage.tsx` — Section 7: "Recommendation" — render the top recommendation as a highlighted card: "Recommended: {vessel_class} at {port_name}" with `total_score` prominently displayed — render all other feasible options as a ranked list below → **(Mahima — Frontend Developer)**

Task 241: Add to `AnalysisResultsPage.tsx` — Section 8: "Why this recommendation?" (Explainability Panel) — render 3 horizontal bars, one for each score component — label: "Cost Score (50% weight): {value}", "Confidence Score (30% weight): {value}", "Coverage Fit Score (20% weight): {value}" — show the data source string under each bar — use `Recharts BarChart` component → **(Mahima — Frontend Developer)**

Task 242: Add to `AnalysisResultsPage.tsx` — Section 9: "Your Decision" — render two buttons: "Accept Recommendation" and "Override & Choose Different Option" — if Override is selected, show a dropdown of all feasible options and a free-text reason input — both buttons call `submitDecision()` — after submission, show a green success card: "Decision recorded and saved to audit log" → **(Mahima — Frontend Developer)**

Task 243: Add to `AnalysisResultsPage.tsx` — Section 10: "Emergency Mode" — a toggle switch labeled "Emergency Procurement Mode" — when toggled on, show a warning badge "Recommendation will not suggest waiting" and call `createAnalysis` again with `is_emergency = true` flag, replacing the current results → **(Mahima — Frontend Developer)**

Task 244: Add to `AnalysisResultsPage.tsx` — Section 11: "Spot vs. COA Comparison" — a collapsible section showing the comparison table: Spot (per voyage) vs. COA (per voyage) vs. Total for N voyages — include a note badge: "COA discount is a configurable assumption, not a live market rate" → **(Mahima — Frontend Developer)**

Route Map Component

Task 245: Create file `frontend/src/components/RouteMap.tsx` — import `MapContainer`, `TileLayer`, `Marker`, `Polyline`, `Popup` from `react-leaflet` → **(Mahima — Frontend Developer)**

Task 246: Add to `RouteMap.tsx` — render a `MapContainer` with `zoom=4` and center at `[15.0, 80.0]` (Bay of Bengal region) — use `OpenStreetMap` free tile layer URL: `https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png` → **(Mahima — Frontend Developer)**

Task 247: Add to `RouteMap.tsx` — render a `Marker` at origin coordinates with a `Popup` showing the port name — use a distinct icon color for origin vs. destination → **(Mahima — Frontend Developer)**

Task 248: Add to `RouteMap.tsx` — render a `Marker` at destination coordinates with a

Popup showing the port name and its constraint values (draft, DWT) → **(Mahima — Frontend Developer)**

Task 249: Add to `RouteMap.tsx` — render `Marker` components for all 4 verified ports (Paradip, Dhamra, Gangavaram, Haldia) using their hardcoded coordinates — each Popup shows the port's max draft and DWT limit → **(Mahima — Frontend Developer)**

Task 250: Add to `RouteMap.tsx` — render a `Polyline` connecting origin coordinates to destination coordinates — line color: blue — add a route label showing the derived distance in nautical miles with note "(approximate great-circle distance)" → **(Mahima — Frontend Developer)**

Task 251: Add `RouteMap` component to `AnalysisResultsPage.tsx` between Section 1 (Context Summary) and Section 2 (Forecast) → **(Mahima — Frontend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Navigate to a completed analysis results page — the map renders showing origin, destination, and the route line
- ☐ All 4 port markers are visible on the map
- ☐ Clicking a marker shows the correct popup

Remaining Pages

Task 252: Edit `frontend/src/pages/ScenarioViewPage.tsx` — load the analysis, render the 5 variable sliders/inputs (quantity $\pm 10\%$, delivery window ± 7 days, vessel class override, port override) — add a "Run Scenario" button that calls `POST / analyses` with the modified values — render a side-by-side comparison table when both analyses are loaded → **(Mahima — Frontend Developer)**

Task 253: Edit `frontend/src/pages/AdminReferencePage.tsx` — render a tabbed view with tabs: Ports | Vessel Classes | Cargo Types | Plants — the Ports tab shows an editable table with Save buttons per row — on save, call `updatePort(id, payload)` → **(Mahima — Frontend Developer)**

Task 254: Edit `frontend/src/pages/AdminUsersPage.tsx` — render a user table with columns: Email | Role | Status | Actions — Actions column has a "Deactivate" button for active users — add a "Create User" form above the table → **(Mahima — Frontend Developer)**

Task 255: Edit `frontend/src/pages/AuditLogPage.tsx` — render a filterable table of audit logs with columns: Timestamp | User | Action | Record ID | Detail — add date range, user, and action type filter inputs → **(Mahima — Frontend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Full user flow test: Login → Dashboard → New Analysis (fill all 5 fields) → Submit → Results page shows all 11 sections → Accept Recommendation →

"Decision recorded" confirmation appears

☐ No console errors in the browser during the full flow

GIT CHECKPOINT — Commit & Push

```
git add .
git commit -m "feat(frontend): all 10 pages, map component, API layer, au
git push origin feat/frontend-ui
```

PR CHECKPOINT — Open Pull Request

- **Branch:** `feat/frontend-ui` → `main`
- **PR Title:** `feat(Phase 2A): Complete Frontend UI – All 10 Pages – Palak`
- **Before merging, verify:**
 - ☐ Task 232 verified: New Analysis form submits and navigates to results 
 - ☐ Task 241 verified: Explainability bar chart renders with correct weights 
 - ☐ Task 250 verified: Route map shows polyline with distance note 
- Run: `git checkout main && git pull origin main`

MERGE PROCEDURE

```
git checkout main && git pull origin main
git merge --squash feat/frontend-ui
git commit -m "feat(frontend): complete all 10 pages and map component"
git push origin main
```

LIKELY CONFLICT FILES:

- `frontend/src/App.tsx` — route additions
- `frontend/src/types/index.ts` — type additions

 **CONFLICT RESOLUTION:**  Keep ALL Route elements in `App.tsx`. Keep ALL interfaces in `types/index.ts`.

PHASE 2B: DEMO DATA + SEED SCRIPTS (Hours 32–36, after Phase 2C is merged) — Param — DevOps + Backend

Goal: Seed all demo scenarios, verify the Golden Demo flow end-to-end with seeded data. Do not start this phase until Task 355 has been merged — the reset script truncates the Phase 2C tables.

Param owns all seed scripts and the reset procedure.

Task 256: Run `git checkout main && git pull origin main && git checkout -b feat/demo-seed` → **(Param — DevOps + Backend)**

Demo Scenario Seed

Task 257: Edit `scripts/seed_demo_scenarios.py` — add a function `create_demo_user()` that inserts a demo user with email `demo@sail.gov.in`, role `PROCUREMENT_OFFICER`, password `SailDemo2026!` if not already exists → **(Param — DevOps + Backend)**

Task 258: Add to `scripts/seed_demo_scenarios.py` — add function `seed_golden_demo_analysis()` that inserts a pre-completed `Analysis` record with: `cargo=coking_coal`, `quantity=75000`, `origin="Newcastle, AU"`, `destination=Paradip`, `delivery_start=2026-11-01`, `delivery_end=2026-11-30` — `status=COMPLETE` — labeled `GENERATED DEMO DATA` in the seed docstring → **(Param — DevOps + Backend)**

Task 259: Add to `scripts/seed_demo_scenarios.py` — add function `seed_forecast_result()` that inserts a `ForecastResult` record for the golden demo analysis with: `p10=18.5`, `p50=22.3`, `p90=27.8`, `arima_baseline=23.1`, `confidence_label="MEDIUM"`, `model_used="LightGBM_quantile_ensemble"` → **(Param — DevOps + Backend)**

Task 260: Add to `scripts/seed_demo_scenarios.py` — add function `seed_feasibility_results()` that inserts `FeasibilityResult` records for the golden demo analysis — Panamax: all pass (`feasible=True`) — Capesize: `draft_pass=False` (`draft 18.2m > Paradip 16.5m limit`), `feasible=False`, `failure_reason="Draft 18.2m exceeds Paradip max draft 16.5m"` — Supramax: `dwt_pass=False` (`50,000-65,000 DWT < 75,000 MT needed`), `feasible=False` — Handysize: `dwt_pass=False`, `feasible=False` → **(Param — DevOps + Backend)**

Task 261: Add to `scripts/seed_demo_scenarios.py` — add function `seed_landed_cost()` that inserts a `LandedCost` record for the golden demo: `freight_rate=22.3`, `baf=1.2`, `usd_inr=83.5`, `total_usd_per_mt=23.5`, `total_inr_per_mt=1962.25`, `total_inr=147168750.0` → **(Param — DevOps + Backend)**

Task 262: Add to `scripts/seed_demo_scenarios.py` — add function `seed_risk_results()` that inserts 6 `RiskResult` records for the golden demo: `freight_volatility=MEDIUM`, `port_draft=LOW`, `delivery_window=LOW`, `bunker_volatility=LOW`, `vessel_availability=NOT_ASSESSED`, `geopolitical=NOT_ASSESSED` → **(Param — DevOps + Backend)**

Task 263: Add to `scripts/seed_demo_scenarios.py` — add function `seed_recommendation()` that inserts a `Recommendation` record: `rank=1`, `vessel_class=Panamax`, `port=Paradip`, `cost_score=0.78`, `confidence_score=0.6`, `coverage_fit_score=0.92`, `total_score=0.756` → **(Param — DevOps + Backend)**

Task 264: Add to `scripts/seed_demo_scenarios.py` — add function

`seed_stockout_alert()` that inserts a stock-out scenario: `current_stock_mt=12000`, `daily_consumption_mt=800`, `days_to_stockout=15`, `days_to_best_window=22`, `is_at_risk=True` → **(Param — DevOps + Backend)**

Task 265: Add to `scripts/seed_demo_scenarios.py` — add function `seed_past_decisions_for_regret()` that inserts 3 past `DecisionRecord` + `RegretScore` pairs with varied regret values (0.5%, 3.2%, 8.1%) to demonstrate the regret score feature → **(Param — DevOps + Backend)**

Task 266: Add to `scripts/seed_demo_scenarios.py` — add function `seed_disruption_alert()` that inserts one active `DisruptionAlert` record with `keyword="Red Sea"`, `headline_text="Red Sea shipping disruptions continue amid ongoing security concerns"` (labeled GENERATED DEMO DATA) → **(Param — DevOps + Backend)**

Task 267: Add `if __name__ == "__main__":` block calling all seed functions in dependency order → **(Param — DevOps + Backend)**

Task 268: Run `cd backend && python ../scripts/seed_demo_scenarios.py` → **(Param — DevOps + Backend)**

Demo Reset Script

Task 269: Edit `scripts/reset_demo_db.py` — add function `reset_to_clean_state()` that runs one `TRUNCATE ... RESTART IDENTITY CASCADE` for, in this order: `quote_requests`, `vendor_quotes`, `bookings`, `cargo_requests`, `password_reset_tokens`, `audit_logs`, `regret_scores`, `decision_records`, `recommendations`, `risk_results`, `landed_costs`, `feasibility_results`, `forecast_results`, `enrichment_cache`, `context_objects`, `disruption_alerts`, `analyses`, `users`, then calls all seed functions from both seed scripts (including Phase 2C seeds) → **(Param — DevOps + Backend)**

Task 270: Add to `scripts/reset_demo_db.py` — print a confirmation message: "Demo database reset complete. Ready for fresh demo." → **(Param — DevOps + Backend)**

✓ VERIFICATION CHECKPOINT

- ☐ Run `python scripts/reset_demo_db.py` — completes without errors
- ☐ Login as `demo@sail.gov.in` — Dashboard shows the seeded golden demo analysis
- ☐ Click the golden demo analysis — Results page shows all sections populated
- ☐ The Capesize row shows  with `failure_reason "Draft 18.2m exceeds Paradip max draft 16.5m"`
- ☐ Stock-out alert shows with red border (`is_at_risk=True`)
- ☐ Red Sea disruption alert appears in the dashboard banner

GIT CHECKPOINT — Commit & Push

```
git add .
git commit -m "feat(demo): complete demo seed scripts and reset procedure"
git push origin feat/demo-seed
```

PR CHECKPOINT — Open Pull Request

- **Branch:** `feat/demo-seed` → `main`
- **PR Title:** `feat(Phase 2B): Demo Seed Data + Reset Script – Ashish`
- **Before merging, verify:**
 - ☐ Task 268 verified: seed runs without errors 
 - ☐ Task 270 verified: reset script prints success message 
- Run: `git checkout main && git pull origin main`

MERGE PROCEDURE

```
git checkout main && git pull origin main
git merge --squash feat/demo-seed
git commit -m "feat(demo): seed scripts and reset procedure"
git push origin main
```

LIKELY CONFLICT FILES:

- `backend/app/main.py` — unlikely but possible if Palak added a new route

 **CONFLICT RESOLUTION:**  Keep all route registrations.

PHASE 3: INTEGRATION + POLISH (Hours 36–44) — All 6 Members

Goal: End-to-end flow verified with real backend + frontend, all data labels visible, all edge cases handled.

Ashish coordinates. Each member tests the features they built. See per-task ownership below.

Task 271: Run `git checkout main && git pull origin main && git checkout -b feat/integration-polish` → **(Ashish — PM + Backend Lead)**

Integration Verification

Task 272: Both members run the full stack locally: `cd backend && uvicorn app.main:app --reload` AND `cd frontend && npm run dev` — verify both start without errors → **(All Members)**

Task 273: Ashish runs `python scripts/reset_demo_db.py` to start from a clean state → **(Ashish — PM + Backend Lead)**

Task 274: Palak navigates to `http://localhost:5173/login` and logs in as `demo@sail.gov.in` — verifies Dashboard loads with the seeded analysis → **(Prachi — Frontend Developer)**

Task 275: Palak clicks "New Analysis" — fills in: coking coal, 75000 MT, "Newcastle, AU", Paradip, delivery 2026-11-01 to 2026-11-30 — submits — verifies navigation to results page → **(Prachi — Frontend Developer)**

Task 276: Palak verifies on the results page: map renders, Capesize shows  with reason, Panamax shows , explainability bar chart renders with 3 bars summing to 1.0 → **(Mahima — Frontend Developer)**

Task 277: Palak clicks "Accept Recommendation" — verifies "Decision recorded" confirmation appears → **(Prachi — Frontend Developer)**

Task 278: Ashish navigates to `http://localhost:8000/audit-logs` — verifies the login and decision events appear in the log → **(Ashish — PM + Backend Lead)**

Edge Case Handling

Task 279: Palak tests: submit New Analysis with `quantity_mt = 250000` (exceeds all vessel classes) — verify the Feasibility section shows all 4 classes as not feasible with a clear message "No vessel class can accommodate this quantity" → **(Prachi — Frontend Developer)**

Task 280: Palak tests: submit New Analysis with `destination = Haldia` and `quantity_mt = 35000` (Handysize) — verify the Haldia result shows `requires_lightering=True` with the lightering note → **(Prachi — Frontend Developer)**

Task 281: Ashish tests: temporarily set `SHIP_AND_BUNKER_BASE_URL` to a broken URL — verify the backend returns a result with `BAF=0` and a "data unavailable" label, rather than crashing → **(Ashish — PM + Backend Lead)**

Task 282: Palak verifies: if backend returns a field as null (e.g., weather flag unavailable), the frontend shows "unavailable" in that section rather than crashing → **(Mahima — Frontend Developer)**

Data Classification Labels Audit

Task 283: Palak does a visual audit of every label on the Results page — verify each data point shows exactly one of: `REAL DATA`, `VERIFIED EXTERNAL DATA`, `DERIVED`, `MODEL OUTPUT`, `ENGINEERING ASSUMPTION`, `GENERATED DEMO DATA` — flag any unlabeled number → **(Mahima — Frontend Developer)**

Task 284: Ashish does a code audit — search for any hardcoded number in the engine files that doesn't have a comment explaining its source or labeling it as ENGINEERING ASSUMPTION → **(Ashish — PM + Backend Lead)**

Task 285: Palak verifies: the explainability bar chart shows $(0.5 \times \text{cost_score}) + (0.3 \times \text{confidence_score}) + (0.2 \times \text{coverage_fit_score})$ weights and that the weights sum to 1.0 in the tooltip — no rounding errors → **(Mahima — Frontend Developer)**

Performance Sanity Check

Task 286: Ashish times the `POST /analyses` endpoint using curl with the golden demo payload — records the time — target: under 30 seconds (acceptable for hackathon, no optimization needed) → **(Ashish — PM + Backend Lead)**

Task 287: Palak verifies the frontend shows the loading message during the analysis creation call so users know the system is working → **(Prachi — Frontend Developer)**

Mobile Responsiveness (Minimal)

Task 288: Palak checks the Results page on a narrow viewport (375px width in Chrome DevTools) — ensures no horizontal overflow, tables are horizontally scrollable rather than cut off → **(Mahima — Frontend Developer)**

Task 289: Add Tailwind `overflow-x-auto` class to all table wrapper divs that might overflow on mobile → **(Mahima — Frontend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Full end-to-end happy path works: Login → New Analysis → Results → Decision
- ☐ 16-page pass from `Features.md`: Landing, Sign Up, Login, Forgot Password, Dashboard counts, History filters, Results (PDF + extras + scenario), Decision approve/reject, Booking manual statuses, Demand merge, Vendor Quotes sample banner, Live Map generated ship, Admin edit + change role, Audit log filters
- ☐ Edge cases tested: oversized quantity, Haldia lightering, broken connector fallback, merge of two different ports rejected, officer cannot self-approve, booking not created from a PENDING decision
- ☐ All data labels visible and correct
- ☐ Vendor Quotes banner and Live Map banner cannot be dismissed
- ☐ No unhandled console errors in browser

🚀 GIT CHECKPOINT — Commit & Push

```
git add .
git commit -m "feat(integration): end-to-end verified, edge cases handled"
git push origin feat/integration-polish
```

PR CHECKPOINT — Open Pull Request

- **Branch:** `feat/integration-polish` → `main`
- **PR Title:** `feat(Phase 3): Integration + Polish — Both`
- **Before merging, verify:**
 - ☐ Task 277 verified: Accept Recommendation flow works end-to-end 
 - ☐ Task 283 verified: all data points have classification labels 
 - ☐ Task 285 verified: score weights sum to 1.0 
- **Run:** `git checkout main && git pull origin main`

MERGE PROCEDURE

```
git checkout main && git pull origin main
git merge --squash feat/integration-polish
git commit -m "feat(integration): end-to-end verified and polished"
git push origin main
```

LIKELY CONFLICT FILES:

- None expected — this phase uses existing files mostly as read/test

 **CONFLICT RESOLUTION:**  Standard rebase if any conflicts arise.

PHASE 4: DEPLOYMENT + DEMO PREP (Hours 44–48) — Param + Prachi + Ashish

Goal: Deployed and accessible URL, demo run-through rehearsed, all checklists complete.

Param handles backend deployment on Render. Prachi handles frontend build + deploy. Ashish prepares demo cheat sheet and verifies end-to-end.

Task 290: Run `git checkout main && git pull origin main && git checkout -b feat/deployment` → **(Param — DevOps + Backend)**

Deployment

Task 291: Ashish creates a new project on Render.com (free tier) → **(Param — DevOps + Backend)**

Task 292: Ashish creates a PostgreSQL database on Render (free tier, 90-day limit) →

(Param — DevOps + Backend)

Task 293: Ashish sets all 13 environment variables in the Render backend service settings, using the values from `backend/.env` → (Param — DevOps + Backend)

Task 294: Ashish sets `DATABASE_URL` in Render to the Render PostgreSQL connection string → (Param — DevOps + Backend)

Task 295: Ashish configures the Render backend service: Build Command = `pip install -r requirements.txt`, Start Command = `uvicorn app.main:app --host 0.0.0.0 --port $PORT` → (Param — DevOps + Backend)

Task 296: Ashish triggers a manual deploy on Render — waits for it to succeed → (Param — DevOps + Backend)

Task 297: Ashish verifies: navigate to `https://<render-url>/health` — returns `{"status": "ok"}` → (Param — DevOps + Backend)

Task 298: Ashish SSHs into Render shell (or uses Render console) to run `alembic upgrade head` → (Param — DevOps + Backend)

Task 299: Ashish runs `python scripts/seed_reference_data.py` and `python scripts/seed_demo_scenarios.py` against the production database → (Param — DevOps + Backend)

Task 300: Palak edits `frontend/.env.production` — set `VITE_API_BASE_URL` to the Render backend URL → (Prachi — Frontend Developer)

Task 301: Palak runs `cd frontend && npm run build` — verifies `dist/` folder is created without errors → (Prachi — Frontend Developer)

Task 302: Palak creates a new static site on Render (or Vercel) pointing to the `frontend/dist` folder → (Prachi — Frontend Developer)

Task 303: Palak navigates to the deployed frontend URL — verifies login works against the deployed backend → (Prachi — Frontend Developer)

✓ VERIFICATION CHECKPOINT

- ☐ Frontend is accessible at a public HTTPS URL
- ☐ Login works on the deployed site
- ☐ New Analysis can be submitted from the deployed site and returns results

Demo Script Run-Through

Task 304: Ashish runs `python scripts/reset_demo_db.py` on the production database — verifies clean state → (Param — DevOps + Backend)

Task 305: Both members do a timed full demo run-through against the deployed URL

— target: 3 minutes → **(All Members)**

Task 306: Palak notes any UI element that causes confusion during the run-through and makes exactly one targeted fix per confusion point → **(Prachi — Frontend Developer)**

Task 307: Ashish prepares a printed cheat sheet with the demo login credentials, the reset command, and the demo flow — one copy per member. The flow must include Demand Board (merge the two seeded plant requests) and Vendor Quotes (read the sample-data banner out loud) → **(Ashish — PM + Backend Lead)**

Final Quality Checks

Task 308: Both members independently verify: the Capesize rejection shows the exact failure reason "Draft 18.2m exceeds Paradip max draft 16.5m" — not a generic message → **(All Members)**

Task 309: Both members independently verify: the recommendation score on the golden demo shows `total_score=0.756` deterministically on every run — no random variation → **(All Members)**

Task 310: Palak verifies: the "ENGINEERING ASSUMPTION" label appears next to the BAF surcharge value — not hidden, not unlabeled → **(Mahima — Frontend Developer)**

Task 311: Palak verifies: the explainability bar chart shows all 3 bars with their weights labeled (50%, 30%, 20%) → **(Mahima — Frontend Developer)**

Task 312: Ashish verifies: after clicking "Accept Recommendation", a new row appears in the `audit_log` table with `action_type="DECISION_RECORDED"` → **(Ashish — PM + Backend Lead)**

Task 313: Both members verify: the stock-out alert on the golden demo shows red border with message "Stock will last 15 days. Next favorable rate window is 22 days away. Book now — cannot afford to wait." → **(All Members)**

Task 314: Palak verifies: the disruption alert banner on the Dashboard shows "Red Sea" keyword alert → **(Prachi — Frontend Developer)**

Task 315: Ashish runs `python scripts/reset_demo_db.py` one final time — verifies all demo data is back to the clean seeded state → **(Param — DevOps + Backend)**

FINAL VERIFICATION CHECKPOINT

- ☐ Deployed frontend URL is live and accessible
- ☐ Login → New Analysis → Results → Decision flow works end-to-end on the deployed site
- ☐ Golden demo analysis is in the database with deterministic values
- ☐ Capesize shows correct failure reason

- ☐ Stock-out alert shows red border
- ☐ Disruption alert appears on Dashboard
- ☐ Explainability bar chart shows 50/30/20 weights
- ☐ All data points have classification labels
- ☐ Reset script works in under 60 seconds
- ☐ Demo run-through completed in under 3 minutes

GIT CHECKPOINT — Final Commit

```
git add .
git commit -m "feat(deployment): deployed to Render, demo verified end-to-end"
git push origin feat/deployment
git checkout main && git pull origin main
git merge --squash feat/deployment
git commit -m "feat(deployment): production deployment complete — Astitva"
git push origin main
git tag -a v1.0-demo -m "SIH26006 Astitva — Demo-ready build"
git push origin v1.0-demo
```

PHASE 2C: FEATURES.MD BACKEND — NEW PAGES API (Hours 18–32) — Ashish — Backend Lead

Goal: All backend API endpoints required for the 8 remaining Features.md pages: Landing, Sign Up, Forgot Password, Decision Record, My Analyses/History, Booking, Demand Board, Vendor Quotes, and Live Map.

Ashish runs this in parallel with Palak's Phase 2A on `feat/page-features-api`.

Task 316: Run `git checkout main && git pull origin main && git checkout -b feat/page-features-api` → (Ashish — PM + Backend Lead)

Sign Up + Forgot Password

Task 317: Add to `backend/app/routers/auth.py` — implement `POST /auth/register` endpoint accepting `UserCreate` schema (Full Name, email/employee-ID, password, confirm_password, role), validates password match, hashes password, inserts into `users` table with `is_active = False` (pending admin approval), returns `{message: "Account created. Await admin approval."}` → (Om — Backend Developer)

Task 318: Create file `backend/app/models/password_reset_token.py` — define SQLAlchemy `PasswordResetToken` model with columns: `id` (Integer PK autoincrement), `user_id` (UUID FK → `users.id`), `token_hash` (String, unique), `expires_at` (DateTime), `used` (Boolean, default False) → (Om — Backend Developer)

Task 319: Add to `backend/app/models/__init__.py` — import `PasswordResetToken` → (Om — Backend Developer)

Task 320: Run `alembic revision --autogenerate -m "add_password_reset_tokens_table"` and `alembic upgrade head` → (Om — Backend Developer)

Task 321: Add to `backend/app/routers/auth.py` — implement `POST /auth/forgot-password` endpoint accepting `{email: str}`, generates a random 6-digit code, stores its hash + 15-minute expiry in `password_reset_tokens`, returns `{reset_code: str}` in the response body (no email service — display the code directly per DO-NOT-BUILD rule) → (Om — Backend Developer)

Task 322: Add to `backend/app/routers/auth.py` — implement `POST /auth/reset-password` accepting `{token: str, new_password: str}`, verifies the token hash is not used and not expired, updates the user's `password_hash`, marks the token `used = True` → (Om — Backend Developer)

✓ VERIFICATION CHECKPOINT

- ☐ `POST /auth/register` with valid payload → 201, user row in DB with `is_active=False`
- ☐ `POST /auth/forgot-password` with known email → returns 6-digit `reset_code` in response
- ☐ `POST /auth/reset-password` with that code → 200; old password no longer works

Decision Record Endpoints

Task 323: Add to `backend/app/routers/analyses.py` — implement `GET /analyses/{id}/decision` returning the `DecisionRecord` for the given analysis, or 404 if none exists → (Ashish — PM + Backend Lead)

Task 324: Add to `backend/app/routers/analyses.py` — implement `PATCH /analyses/{id}/decision/approve` — role-check: only `PLANT_MANAGER` may call this — sets a `manager_approved` boolean field on the `DecisionRecord` to `True`, logs `DECISION_APPROVED` to audit log → (Ashish — PM + Backend Lead)

Task 325: Add `manager_approved` (Boolean, nullable, default `None`) and `reject_reason` (Text, nullable) columns to `backend/app/models/decision_record.py` — run `alembic revision --autogenerate -m "decision_approval_fields"` and `alembic upgrade head` → (Ashish — PM + Backend Lead)

Task 326: Add to `backend/app/routers/analyses.py` — implement `PATCH /analyses/{id}/decision/reject` — role-check: only `PLANT_MANAGER` — accepts `{reason: str}` body, sets `manager_approved = False`, stores `reject_reason`, logs `DECISION_REJECTED` to audit log → (Ashish — PM + Backend Lead)

Task 327: Add to `backend/app/models/schemas.py` — define `DecisionRecordResponse` schema with all `DecisionRecord` fields including `manager_approved`, `reject_reason`, `decided_at`, `decided_by` → **(Ashish — PM + Backend Lead)**

✓ VERIFICATION CHECKPOINT

- ☐ GET `/analyses/{id}/decision` after submitting a decision → returns decision record
- ☐ PATCH approve with OFFICER role → 403
- ☐ PATCH approve with MANAGER role → 200, `manager_approved=True` in DB

My Analyses / History — Filtering

Task 328: Extend GET `/analyses` in `backend/app/routers/analyses.py` — add optional query params: `cargo_type` (str), `port_id` (int), `date_from` (date), `date_to` (date), `status` (str) — apply as SQL filters using SQLAlchemy `.filter()` → **(Ashish — PM + Backend Lead)**

Task 329: Add to `backend/app/models/schemas.py` — define `AnalysisListItem` schema with fields: `id`, `created_at`, `cargo_type`, `origin_port`, `destination_port_name`, `quantity_mt`, `status` — used for the history list (lighter payload than `FullAnalysisResponse`) → **(Ashish — PM + Backend Lead)**

Task 330: Update GET `/analyses` to return `list[AnalysisListItem]` instead of `list[Analysis]` ORM objects → **(Ashish — PM + Backend Lead)**

✓ VERIFICATION CHECKPOINT

- ☐ GET `/analyses?cargo_type=coking_coal` returns only coking coal analyses
- ☐ GET `/analyses?date_from=2026-11-01` returns only analyses from that date onwards

Booking Backend

Task 331: Create file `backend/app/models/booking.py` — define SQLAlchemy `Booking` model with columns: `id` (UUID PK), `decision_record_id` (UUID FK → `decision_records.id`, unique), `status` (String: WAITING/SENT_TO_BROKER/CONFIRMED/CANCELLED), `initiated_by` (UUID FK → `users.id`), `initiated_at` (DateTime, server_default now), `confirmed_by` (UUID FK → `users.id`, nullable), `confirmed_at` (DateTime, nullable), `note` (Text, nullable) → **(Om — Backend Developer)**

Task 332: Import `Booking` in `backend/app/models/__init__.py` → **(Om — Backend Developer)**

Task 333: Run `alembic revision --autogenerate -m "add_bookings_table"` and `alembic upgrade head` → (Om — Backend Developer)

Task 334: Create file `backend/app/routers/bookings.py` — define `APIRouter` with prefix `/bookings`, protected by `get_current_user` → (Om — Backend Developer)

Task 335: Add to `backend/app/routers/bookings.py` — implement `POST /bookings` accepting `{decision_record_id: UUID}` — validates the decision exists and `manager_approved = True` (cannot book an unapproved decision) — creates a `Booking` record with `status = WAITING` — logs `BOOKING_INITIATED` to audit log → (Om — Backend Developer)

Task 336: Add to `backend/app/routers/bookings.py` — implement `GET /bookings/{id}` returning the booking with its current status → (Om — Backend Developer)

Task 337: Add to `backend/app/routers/bookings.py` — implement `PATCH /bookings/{id}/confirm` — role-check: `ADMIN` or `PLANT_MANAGER` only — sets `status = CONFIRMED`, records `confirmed_by` and `confirmed_at`, logs `BOOKING_CONFIRMED` to audit log → (Om — Backend Developer)

Task 338: Add to `backend/app/routers/bookings.py` — implement `PATCH /bookings/{id}/cancel` — sets `status = CANCELLED`, logs `BOOKING_CANCELLED` to audit log → (Om — Backend Developer)

Task 339: Add to `backend/app/models/schemas.py` — define `BookingResponse` schema with all `Booking` fields → (Om — Backend Developer)

Task 340: Register `bookings_router` in `backend/app/main.py` → (Om — Backend Developer)

✓ VERIFICATION CHECKPOINT

- ☐ `POST /bookings` on an unapproved decision → 400 error with clear message
- ☐ `POST /bookings` on approved decision → 201, status=WAITING
- ☐ `PATCH confirm` with `OFFICER` role → 403

Demand Board Backend

Task 341: Create file `backend/app/models/cargo_request.py` — define `SQLAlchemy` `CargoRequest` model with columns: `id` (UUID PK), `plant_id` (Integer FK → `reference_plants.id`), `cargo_type_id` (Integer FK → `reference_cargo_types.id`), `quantity_mt` (Float), `destination_port_id` (Integer FK → `reference_ports.id`), `requested_by` (UUID FK → `users.id`), `status` (String: `OPEN/MERGED/CANCELLED`), `merged_into_id` (UUID FK → `cargo_requests.id`, nullable, self-referential), `created_at` (DateTime, server_default now) → (Om — Backend Developer)

Task 342: Import `CargoRequest` in `backend/app/models/__init__.py` → (Om — Backend Developer)

Task 343: Run `alembic revision --autogenerate -m "add_cargo_requests_table"` and `alembic upgrade head` → (Om — Backend Developer)

Task 344: Create file `backend/app/routers/demand.py` — define `APIRouter` with prefix `/demand`, protected by `get_current_user` → (Om — Backend Developer)

Task 345: Add to `backend/app/routers/demand.py` — implement `GET /demand` returning all `CargoRequest` records with `status = OPEN`, joined with plant name and port name → (Om — Backend Developer)

Task 346: Add to `backend/app/routers/demand.py` — implement `POST /demand` accepting `CargoRequestCreate` schema (`plant_id`, `cargo_type_id`, `quantity_mt`, `destination_port_id`) — creates a new `CargoRequest` with `status = OPEN` → (Om — Backend Developer)

Task 347: Add to `backend/app/routers/demand.py` — implement `POST /demand/merge` accepting `{request_id_a: UUID, request_id_b: UUID}` — validates both requests are `OPEN` and have the same `destination_port_id` (raise 400 if different ports) — sets both records to `status = MERGED`, creates a new `OPEN` `CargoRequest` with `quantity_mt = a.quantity_mt + b.quantity_mt` and `merged_into_id = new_record.id` — logs `DEMAND_MERGED` to audit log → (Om — Backend Developer)

Task 348: Add to `backend/app/models/schemas.py` — define `CargoRequestCreate` and `CargoRequestResponse` schemas → (Om — Backend Developer)

Task 349: Register `demand_router` in `backend/app/main.py` → (Om — Backend Developer)

✓ VERIFICATION CHECKPOINT

- ☐ `GET /demand` returns seeded open requests
- ☐ `POST /demand/merge` with two requests going to Paradip → 200, new merged request created
- ☐ `POST /demand/merge` with requests going to different ports → 400

Vendor Quotes Backend

Task 350: Create file `backend/app/models/vendor_quote.py` — define SQLAlchemy `VendorQuote` model with columns: `id` (Integer PK autoincrement), `quote_request_label` (String), `broker_name` (String), `vessel_type` (String), `quoted_rate_usd_per_mt` (Float), `delivery_days` (Integer), `valid_until` (Date), `is_sample_data` (Boolean, default True) → (Param — DevOps + Backend)

Task 351: Import `VendorQuote` in `backend/app/models/__init__.py` → (Param —

DevOps + Backend)

Task 352: Run `alembic revision --autogenerate -m "add_vendor_quotes_table"` and `alembic upgrade head` → (Param — DevOps + Backend)

Task 353: Create file `backend/app/routers/quotes.py` — define `APIRouter` with prefix `/quotes` → (Param — DevOps + Backend)

Task 354: Add to `backend/app/routers/quotes.py` — implement `GET /quotes` returning all `VendorQuote` records — always include `is_sample_data = true` on every record → (Param — DevOps + Backend)

Task 355: Add to `backend/app/models/schemas.py` — define `VendorQuoteResponse` schema with all `VendorQuote` fields plus a `sample_data_notice` string constant: `"These quotes are sample data. Real broker integration is not connected."` → (Param — DevOps + Backend)

Task 356: Register `quotes_router` in `backend/app/main.py` → (Param — DevOps + Backend)

Task 357: Add seed function `seed_vendor_quotes()` to `scripts/seed_demo_scenarios.py` — insert 3 sample `VendorQuote` records with varied vessel types, rates around 20–25 USD/MT, and `is_sample_data = True` → (Param — DevOps + Backend)

✓ VERIFICATION CHECKPOINT

- ☐ `GET /quotes` returns 3 seeded sample records
- ☐ Every record has `is_sample_data = true`

Live Map — Disruption Alerts Endpoint

Task 358: Create file `backend/app/routers/map.py` — define `APIRouter` with prefix `/map` → (Param — DevOps + Backend)

Task 359: Add to `backend/app/routers/map.py` — implement `GET /map/route` accepting query params `origin_locode` and `destination_locode` — calls `locode_connector.get_port_coordinates` for both, returns `{origin_lat, origin_lon, destination_lat, destination_lon, estimated_distance_nm, note: "great-circle approximation"}` → (Param — DevOps + Backend)

Task 360: Add to `backend/app/routers/map.py` — implement `GET /map/ship-position` accepting `origin_lat, origin_lon, destination_lat, destination_lon, progress_pct (0.0–1.0)` — returns a linearly interpolated lat/lon position with note: `"Generated position – not a live AIS feed"` → (Param — DevOps + Backend)

Task 361: Add to `backend/app/routers/analyses.py` — implement `GET /disruption-alerts` returning all `DisruptionAlert` records with `is_active = True` —

used by the Dashboard alerts section → (Param — DevOps + Backend)

Task 362: Register `map_router` in `backend/app/main.py` → (Param — DevOps + Backend)

Task 363: Update `docs/API_CONTRACT.md` — add all new endpoints from Phase 2C: `/auth/register`, `/auth/forgot-password`, `/auth/reset-password`, `/analyses/{id}/decision`, `/analyses/{id}/decision/approve`, `/analyses/{id}/decision/reject`, `/bookings`, `/demand`, `/demand/merge`, `/quotes`, `/map/route`, `/map/ship-position`, `/disruption-alerts` → (Param — DevOps + Backend)

✓ VERIFICATION CHECKPOINT

- ☐ GET `/map/route?origin_locode=AUNEW&destination_locode=INPDI` returns both lat/lon pairs and distance
- ☐ GET `/map/ship-position?...&progress_pct=0.5` returns midpoint coordinates
- ☐ GET `/disruption-alerts` returns the seeded Red Sea alert

🚀 GIT CHECKPOINT — Commit & Push

```
git add .
git commit -m "feat(api): all new-page endpoints — signup, decision, booking, demand, quotes, map"
git push origin feat/page-features-api
```

✂️ PR CHECKPOINT — Open Pull Request

- **Branch:** `feat/page-features-api` → `main`
- **PR Title:** `feat(Phase 2C): New-Pages Backend API — Signup, Booking, Demand, Quotes, Map — Ashish`
- **Before merging, verify:**
 - ☐ Task 322 verified: forgot-password + reset flow works end-to-end ✓
 - ☐ Task 347 verified: demand merge rejects different-port requests ✓
 - ☐ Task 360 verified: ship-position returns interpolated lat/lon ✓
- Run: `git checkout main && git pull origin main`

✂️ MERGE PROCEDURE

```
git checkout main && git pull origin main
git merge --squash feat/page-features-api
git commit -m "feat(api): complete new-pages backend — signup, booking, demand, quotes, map"
git push origin main
```

⚠️ LIKELY CONFLICT FILES:

- `backend/app/main.py` — multiple new router registrations
- `backend/app/models/schemas.py` — new schemas added by Palak and

Ashish simultaneously



CONFLICT RESOLUTION: ● Keep ALL `include_router` lines. Keep ALL schema classes — never delete one added by the other member.



PHASE 2D: FEATURES.MD FRONTEND — NEW PAGES UI (Hours 28–36) — Palak — Frontend Lead

Goal: Build the 8 remaining Features.md pages: Landing, Sign Up, Forgot Password, Decision Record, My Analyses/History, Booking, Demand Board, Vendor Quotes, Live Map.

Palak starts this phase only after Phase 2C is merged to main.

Task 364: Run `git checkout main && git pull origin main && git checkout -b feat/page-features-ui` → **(Prachi — Frontend Developer)**

Route Registration (App.tsx)

Task 365: Edit `frontend/src/App.tsx` — add new routes (outside `ProtectedRoute` where public, inside where auth required): `/` (LandingPage, public), `/signup` (SignUpPage, public), `/forgot-password` (ForgotPasswordPage, public), `/analyses/:id/decision` (DecisionRecordPage, protected), `/history` (HistoryPage, protected), `/bookings/:id` (BookingPage, protected), `/demand` (DemandBoardPage, protected), `/quotes` (VendorQuotesPage, protected), `/map` (LiveMapPage, protected) → **(Prachi — Frontend Developer)**

API Layer Additions

Task 366: Add to `frontend/src/api/auth.ts` — define `register(payload: RegisterPayload): Promise<{message: string}>`, `forgotPassword(email: string): Promise<{reset_code: string}>`, `resetPassword(token: string, new_password: string): Promise<{message: string}>` → **(Prachi — Frontend Developer)**

Task 367: Create file `frontend/src/api/bookings.ts` — define `createBooking(decision_record_id: string): Promise<BookingResponse>`, `getBooking(id: string): Promise<BookingResponse>`, `confirmBooking(id: string): Promise<BookingResponse>`, `cancelBooking(id: string): Promise<BookingResponse>` → **(Prachi — Frontend Developer)**

Task 368: Create file `frontend/src/api/demand.ts` — define `listDemand(): Promise<CargoRequestResponse[]>`, `postDemand(payload: CargoRequestCreate): Promise<CargoRequestResponse>`, `mergeDemand(a: string, b: string): Promise<CargoRequestResponse>` → **(Prachi — Frontend Developer)**

Task 369: Create file `frontend/src/api/quotes.ts` — define `listQuotes():`

`Promise<VendorQuoteResponse[]>` → **(Prachi — Frontend Developer)**

Task 370: Create file `frontend/src/api/map.ts` — define `getRoute(origin_locode: string, destination_locode: string): Promise<RouteResponse>`, `getShipPosition(origin_lat: number, origin_lon: number, destination_lat: number, destination_lon: number, progress_pct: number): Promise<{lat: number, lon: number, note: string}>` → **(Prachi — Frontend Developer)**

TypeScript Type Additions

Task 371: Add to `frontend/src/types/index.ts` — define interfaces:

`BookingResponse`, `CargoRequestCreate`, `CargoRequestResponse`, `VendorQuoteResponse`, `RouteResponse`, `RegisterPayload`, `DecisionRecordResponse` → **(Prachi — Frontend Developer)**

Landing Page (Page 1)

Task 372: Create file `frontend/src/pages/LandingPage.tsx` — render 5 sections matching `Features.md`: Introduction (one-line headline), Problem (simple numbers card), How It Works (4-step numbered list), What You Get (4 feature bullet cards), Trust/Proof (labelled note on real vs. sample data) → **(Prachi — Frontend Developer)**

Task 373: Add to `LandingPage.tsx` — two prominent buttons at top-right: "Login" (navigates to `/login`) and "Sign Up" (navigates to `/signup`) — page must NOT use `<Layout>` wrapper (no nav bar on public landing) → **(Prachi — Frontend Developer)**

Sign Up Page (Page 2)

Task 374: Create file `frontend/src/pages/SignUpPage.tsx` — render a centered card with inputs: Full Name (text), Email / Employee ID (text), Password (password), Confirm Password (password), Role (dropdown: Officer / Manager / Admin) → **(Prachi — Frontend Developer)**

Task 375: Add to `SignUpPage.tsx` — client-side validation: all fields required, password and confirm password must match, password min 8 characters — show inline error per field — "Create Account" button disabled until all validations pass → **(Prachi — Frontend Developer)**

Task 376: Add to `SignUpPage.tsx` — on submit, call `register(payload)` — on success, show: "Account created. An admin must activate your account before you can log in." with a link back to `/login` — on failure show server error message → **(Prachi — Frontend Developer)**

Forgot Password Page (Page 3 sub-flow)

Task 377: Create file `frontend/src/pages/ForgotPasswordPage.tsx` — render a two-step form: Step 1: email input + "Get Reset Code" button — Step 2 (shown after step 1 succeeds): displays the returned reset code in a visible code box, a new-password input, and a "Reset Password" button → **(Prachi — Frontend Developer)**

Task 378: Add to `ForgotPasswordPage.tsx` — on Step 2 submit, call `resetPassword(code, newPassword)` — on success navigate to `/login` with a toast or inline message "Password reset. Please log in." → **(Prachi — Frontend Developer)**

Task 379: Add a "Forgot Password?" link to `frontend/src/pages/LoginPage.tsx` that navigates to `/forgot-password` → **(Prachi — Frontend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Sign Up form: password mismatch shows inline error, button stays disabled
- ☐ Sign Up submit → success message with "admin must activate" note
- ☐ Forgot Password: step 1 shows code, step 2 resets and redirects to login

Decision Record Page (Page 8)

Task 380: Create file `frontend/src/pages/DecisionRecordPage.tsx` — on mount, call `getAnalysis(id)` to load the decision and analysis — render two sections: "Final Decision Summary" (what was recommended vs. what was chosen, override reason if any) and "Approval Status" (pending / approved / rejected badge) → **(Mahima — Frontend Developer)**

Task 381: Add to `DecisionRecordPage.tsx` — for `PLANT_MANAGER` role only: show "Approve" (green button) and "Reject" (red button) — "Reject" must show a required short-reason text input before submitting — call `approve/reject` API endpoints on click — disable both buttons once a decision has been made (`manager_approved !== null`) → **(Mahima — Frontend Developer)**

Task 382: Add to `DecisionRecordPage.tsx` — "Download Final Record" button that calls `GET /analyses/{id}/export` and triggers a file download (same plain-text export as the Analysis Results page export) → **(Mahima — Frontend Developer)**

Task 383: Add a link from `AnalysisResultsPage.tsx` "Approve / Send for Booking" button to navigate to `/analyses/{id}/decision` after the decision is submitted → **(Mahima — Frontend Developer)**

My Analyses / History Page (Page 9)

Task 384: Create file `frontend/src/pages/HistoryPage.tsx` — on mount, call

`listAnalyses()` — render a table with columns: Date | Cargo Type | Origin → Destination | Quantity | Status — each row has an "Open" link to `/analyses/{id}` → **(Prachi — Frontend Developer)**

Task 385: Add to `HistoryPage.tsx` — filter controls above the table: Date From (date input), Date To (date input), Cargo Type (text input), Port (text input) — on filter change, re-call `listAnalyses(filters)` — show "No analyses match your filters" empty state when the filtered list is empty → **(Prachi — Frontend Developer)**

Task 386: Add "History" nav link to `frontend/src/components/Layout.tsx` nav bar → **(Prachi — Frontend Developer)**

Booking Page (Page 10)

Task 387: Create file `frontend/src/pages/BookingPage.tsx` — load booking by ID from `getBooking(id)` — render a "Booking Status" card showing current status (WAITING / SENT_TO_BROKER / CONFIRMED / CANCELLED) with a timeline-style display → **(Mahima — Frontend Developer)**

Task 388: Add to `BookingPage.tsx` — for ADMIN / PLANT_MANAGER role: show "Mark as Confirmed" button (calls `confirmBooking`) and "Cancel" button (calls `cancelBooking`) — disable both if status is already CONFIRMED or CANCELLED → **(Mahima — Frontend Developer)**

Task 389: Add to `BookingPage.tsx` — render a persistent non-dismissible banner at the top: "Booking status is tracked manually. This system does not connect to any shipping company or broker system." → **(Mahima — Frontend Developer)**

Task 390: Add an "Initiate Booking" button to `DecisionRecordPage.tsx` — visible only if `manager_approved = true` and no booking exists yet — calls `createBooking(decision_record_id)` and navigates to `/bookings/{id}` on success → **(Mahima — Frontend Developer)**

Demand Board Page (Page 11)

Task 391: Create file `frontend/src/pages/DemandBoardPage.tsx` — on mount, call `listDemand()` — render a table of open requests with columns: Plant | Cargo Type | Quantity (MT) | Destination Port | Posted By | Actions → **(Mahima — Frontend Developer)**

Task 392: Add to `DemandBoardPage.tsx` — checkboxes on each row to select requests for merging — "Merge Selected Requests" button enabled only when exactly 2 rows are checked — on click, call `mergeDemand(a, b)` — on success show "Requests merged into one combined request" and refresh the list — on 400 error (different ports) show "Cannot merge: requests go to different ports" → **(Mahima — Frontend Developer)**

Task 393: Add to `DemandBoardPage.tsx` — "Post New Request" button that opens an inline form (not a new page): plant dropdown, cargo type dropdown, quantity, destination port dropdown — on submit calls `postDemand(payload)` — on success refreshes the list → **(Mahima — Frontend Developer)**

Task 394: Add "Demand Board" nav link to `frontend/src/components/Layout.tsx` nav bar (visible to all authenticated roles) → **(Mahima — Frontend Developer)**

Vendor Quotes Page (Page 12)

Task 395: Create file `frontend/src/pages/VendorQuotesPage.tsx` — on mount, call `listQuotes()` — render a table of quotes with columns: Broker | Vessel Type | Rate (\$/MT) | Delivery Days | Valid Until — alongside each quote row, show the system's own P50 forecast for comparison → **(Mahima — Frontend Developer)**

Task 396: Add to `VendorQuotesPage.tsx` — render a persistent non-dismissible banner at the top of the page: "Sample data only. These quotes are shown to demonstrate how real broker replies would appear. No live broker integration is connected." — banner styled in amber/yellow, cannot be dismissed → **(Mahima — Frontend Developer)**

Task 397: Add to `VendorQuotesPage.tsx` — a "Send for Quotes" button that shows a modal: "In the full system, this would notify registered brokers. Currently logged for reference only." — on confirm, shows a toast "Quote request logged." — no actual API call to a broker → **(Mahima — Frontend Developer)**

Live Map Page (Page 13)

Task 398: Create file `frontend/src/pages/LiveMapPage.tsx` — reuse `RouteMap.tsx` component, rendered full-screen — on mount call `getRoute(origin_locode, destination_locode)` using origin/destination from the current analysis context (pass as query params or use a stored analysis) → **(Mahima — Frontend Developer)**

Task 399: Add to `LiveMapPage.tsx` — animated ship position: use `setInterval` every 3 seconds to call `getShipPosition(..., progress_pct)` and increment `progress_pct` by 0.05 each tick (looping back to 0 when it reaches 1.0) — render a distinct ship icon `Marker` at the returned lat/lon — clear interval on component unmount → **(Mahima — Frontend Developer)**

Task 400: Add to `LiveMapPage.tsx` — render a persistent non-dismissible banner: "Ship position is automatically generated to demonstrate tracking. This is not a live AIS feed." — styled in amber/yellow → **(Mahima — Frontend Developer)**

Task 401: Add "Live Map" nav link to `frontend/src/components/Layout.tsx` nav bar → **(Mahima — Frontend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ Live Map: ship icon moves along the route every 3 seconds
- ☐ Live Map banner is visible and cannot be dismissed
- ☐ Vendor Quotes: sample-data banner is always visible
- ☐ Demand Board: merge of two different-port requests shows error message

Seed Data for Phase 2C Tables

Task 402: Add seed function `seed_cargo_requests()` to `scripts/seed_demo_scenarios.py` — insert 2 open `CargoRequest` records: Bhilai plant (coking coal, 40000 MT, Paradip) and Rourkela plant (coking coal, 35000 MT, Paradip) — both `status = OPEN` — these are the requests used in the Demand Board merge demo → **(Param — DevOps + Backend)**

Task 403: Add seed call `seed_cargo_requests()` to the `if __name__ == "__main__":` block in `scripts/seed_demo_scenarios.py` → **(Param — DevOps + Backend)**

✓ VERIFICATION CHECKPOINT

- ☐ GET `/demand` returns the two seeded plant requests
- ☐ Merging them in the UI creates a combined 75,000 MT request for Paradip

Final Page Wiring

Task 404: Verify `frontend/src/App.tsx` has routes for all 16 pages from `Features.md` — confirm each route has the correct page component assigned → **(Mahima — Frontend Developer)**

Task 405: Verify `frontend/src/components/Layout.tsx` nav bar links are correct and visible for appropriate roles — test each nav link navigates without a 404 → **(Mahima — Frontend Developer)**

Task 406: Add role-based nav visibility: "Admin Reference" and "Admin Users" links only visible if `user.role === "ADMIN"` — all other links visible to all authenticated roles → **(Mahima — Frontend Developer)**

Task 407: Palak does a 16-page walkthrough: open each page by navigating to its URL — verify no page throws a runtime error and every page renders at least its main section → **(Mahima — Frontend Developer)**

Task 408: Update `frontend/src/App.tsx` PR title in the commit message to reference all 16 pages covered → **(Mahima — Frontend Developer)**

✓ VERIFICATION CHECKPOINT

- ☐ All 16 pages from Features.md are reachable without errors
- ☐ Public pages (Landing, Sign Up, Forgot Password) are accessible without a token
- ☐ Protected pages redirect to `/login` when accessed without a token
- ☐ Role-only controls (Approve/Reject, Confirm Booking, Admin links) hidden from unauthorized roles

🔥 GIT CHECKPOINT — Commit & Push

```
git add .  
git commit -m "feat(frontend): all 16 Features.md pages – landing, signu  
git push origin feat/page-features-ui
```

✂️ PR CHECKPOINT — Open Pull Request

- **Branch:** `feat/page-features-ui` → `main`
- **PR Title:** `feat(Phase 2D): All 16 Features.md Pages – Palak`
- **Before merging, verify:**
 - ☐ Task 400 verified: Live Map banner cannot be dismissed ✓
 - ☐ Task 392 verified: Demand merge shows error for different-port requests ✓
 - ☐ Task 407 verified: all 16 pages render without runtime errors ✓
- Run: `git checkout main && git pull origin main`

✂️ MERGE PROCEDURE

```
git checkout main && git pull origin main  
git merge --squash feat/page-features-ui  
git commit -m "feat(frontend): complete all 16 pages – Astitva full page  
git push origin main
```

⚠️ LIKELY CONFLICT FILES:

- `frontend/src/App.tsx` — new routes added
- `frontend/src/components/Layout.tsx` — new nav links added
- `frontend/src/types/index.ts` — new interfaces added

💡 **CONFLICT RESOLUTION:** ● Keep ALL Route elements. Keep ALL nav links. Keep ALL interfaces.

📋 MASTER TASK TRACKER

Phase	Tasks	Owner	Hours
Pre-Phase 0: GitHub Setup	G1–G40	Ashish (PM)	Pre-hackathon
Phase 0: Foundation	1–62	Param + Prachi + Ashish	0–4h
Phase 1A: Database + Models	63–91	Om + Param	4–10h
Phase 1B: Backend Core	92–149	Ashish + Om	4–16h
Phase 1C: ML Engines	150–204	Palak	6–18h
Phase 2A: Frontend UI (core 10 pages)	205–255	Prachi + Mahima	18–28h
Phase 2C: New-pages API backend	316–363	Ashish + Om + Param	18–32h (parallel with 2A)
Phase 2D: New-pages frontend (16 total)	364–408	Prachi + Mahima	28–36h (after 2C merges)
Phase 2B: Demo Seed Data + Reset	256–270	Param	32–36h (after 2C merges)
Phase 3: Integration + Polish	271–289	All Members	36–44h
Phase 4: Deployment + Demo Prep	290–315	Param + Prachi + Ashish	44–48h
TOTAL	448 tasks (G1–G40 + Tasks 1–408)		48 hours



DO-NOT-BUILD LIST (Scope Guard)

The following items are explicitly excluded. If either member is tempted to add any of these, stop and check with the other member first.

- ✗ Kafka / message queues – APScheduler is sufficient
- ✗ Redis / caching layer – PostgreSQL enrichment_cache table is sufficient
- ✗ Microservices / Docker Compose – single monolith is sufficient
- ✗ Kubernetes – not needed for a 2-person demo
- ✗ WebSockets / real-time push – no user requires real-time updates
- ✗ Email / SMS notifications – Forgot Password shows the reset code on screen
- ✗ Individual vessel live tracking (AIS) – Live Map moves a generated icon al
- ✗ Live broker or carrier connections – Vendor Quotes and Booking stay manual

- ✗ Real COA rate benchmark – Platts/S&P is paid; use assumption
- ✗ LP/MIP optimizer – deterministic weighted score is sufficient
- ✗ Extra user roles beyond 5 – out of scope
- ✗ Free-text LLM narrative explanation – structured object is sufficient
- ✗ Blockchain – not relevant to this problem

EMERGENCY CONTACTS

If a connector breaks during demo:

- Bunker price: use cached value from enrichment_cache table
- BDRY proxy: fall back to ml/data/bdry_history.csv last row
- Forex: hardcode 83.5 as a fallback constant in config.py
- Weather: return "FAVORABLE" as default flag

If the deployed backend goes down during demo:

- Run locally: `cd backend && uvicorn app.main:app --reload`
- Update frontend .env to point to localhost:8000
- Rebuild frontend: `npm run build && npm run dev`
- Demo from localhost – judges don't require a live deployment

If the database is corrupted:

- Run: `python scripts/reset_demo_db.py`
- This always restores to a clean, known-good demo state